

MANUAL

Técnico



ÍNDICE

1. INTRODUCCIÓN	1
2. OBJETIVOS.....	1
2.1 Objetivo general	2
2.2 Objetivos técnicos	2
2.3. Alcance	3
3. REQUISITOS.....	5
3.1 Requisitos Funcionales.....	5
3.2 Requisitos no Funcionales.....	8
3.3 Requisitos para el Cliente (Usuario Final)	10
3.4 Requisitos para el Servidor de Desarrollo	12
4. HERRAMIENTAS Y STACK TECNOLÓGICO	16
4.1 Frontend: Core y Framework	17
4.2 Estilos e Interfaz de Usuario (UI)	18
4.2.1 Tailwind CSS (Motor de Estilos).....	18
4.2.2 Angular Material (Componentes UI).....	18
4.2.3 Lucide Angular (Iconografía)	18
4.2.4 Modos oscuro	19
4.2.5 Responsive.....	19
4.3 Backend & Persistencia (BaaS).....	19
4.4 Librerías Auxiliares	20
4.5 Testing	25
4.5.1 Karma & Jasmine: Pruebas Unitarias	25
4.5.2 Cypress: Pruebas de Extremo a Extremo (E2E).....	27
4.5.3 Postman: Validación de API y Contratos de Datos.....	28

4.6 Integración con Inteligencia Artificial.....	32
5. GUÍA DE INSTALACIÓN (ENTORNO LOCAL)	35
5.1 Instalación de Prerrequisitos (Paso a Paso).....	35
5.2 Entorno de desarrollo	37
6. GUÍA DE DESPLIEGUE (ENTORNO CLOUD - VERCEL)	38
6.1 Configuración de Build	38
6.2 Gestión de Rutas (SPA).....	39
6.3 Variables de Entorno en Producción	39
7. ARQUITECTURA DE BASE DE DATOS (SUPABASE).....	39
7.1 Aislamiento de Capas y Seguridad de Esquemas.....	39
7.2 Modelo Relacional y Especialización de roles.....	40
7.2.1 Diagrama Entidad-Relación (ERD).....	45
7.3 Lógica de Negocio	51
7.3.1 Lógica de Servidor: Triggers y Funciones RPC	51
7.3.2 Seguridad (Row Level Security - RLS).....	53
7.4 Seguridad Avanzada y Ejecución de Funciones (Security Definer).....	54
8. EJECUCIÓN Y PRUEBAS DEL APLICATIVO.....	55
8.1 Comandos de Ciclo de Vida	55
8.2 Gestión de Bundles y Optimización (Caso de Estudio)	55
9. ARQUITECTURA DEL PROYECTO (ESTRUCTURA DE CARPETAS)	58
10. NAVEGACIÓN Y SEGURIDAD EN RUTAS.....	59
11. Estrategia de Seguridad y Despliegue en Vercel (CI/CD).....	63
12. DOCUMENTACIÓN.....	63
12.1. Documentación Externa (Manuales).....	64
12.2. Documentación Interna del Código (Clean Code y JSDoc).....	64
12.3. Documentación Gráfica y Diagramas	64
12.4. Repositorio, README y Asistencia por IA	64
13. MEJORAS FUTURAS	65

1. INTRODUCCIÓN

El presente manual técnico describe la arquitectura interna, el proceso de instalación y los protocolos de despliegue de **cuidaDos**, una plataforma web (Single Page Application) destinada a la gestión de servicios asistenciales.

El sistema ha sido construido desacoplando el frontend del backend, utilizando una arquitectura *serverless*¹ para la base de datos y un enfoque reactivo en la interfaz de usuario. Este documento está dirigido a desarrolladores y administradores de sistemas que requieran mantener, auditar o escalar la aplicación.

La plataforma centraliza la gestión de los cuidados a través de una interfaz moderna y accesible. Proporciona herramientas avanzadas como un buscador de servicios, un sistema de contratación y calendario, mensajería en tiempo real y un panel de administración con analíticas de uso. Además, como valor añadido e innovador, integra un asistente virtual basado en Inteligencia Artificial (LLM) capaz de resolver las dudas de los usuarios interactuando directamente con la documentación del sistema.

Técnicamente, CuidaDos se apoya en una arquitectura *Serverless* moderna, utilizando **Angular** para la capa de presentación interactiva y **Supabase (PostgreSQL)** como *Backend-as-a-Service* (BaaS), lo que garantiza escalabilidad, seguridad en el manejo de datos (mediante RLS) y una experiencia de usuario en tiempo real.

2. OBJETIVOS

Este manual técnico tiene como objetivo la especificación los componentes y recursos técnicos del sistema al usuario final, con la finalidad de apoyar situaciones de mantenimiento o mejoras aplicadas

¹ Modelo de ejecución en la nube donde la asignación de recursos es dinámica y gestionada por terceros (Supabase y Vercel). En este esquema, la base de datos PostgreSQL actúa como un servicio independiente capaz de exponer sus datos vía API REST y WebSockets sin requerir un servidor intermedio dedicado. Se delega la seguridad y el mantenimiento de la infraestructura, permitiendo una alta disponibilidad y tolerancia a fallos desde el primer despliegue.

2.1 Objetivo general

Diseñar y desarrollar una aplicación web funcional que aporte una solución tecnológica a un problema concreto.

Objetivos Específicos:

- Implementar un sistema para la contratación de servicios empresariales para el cuidado de personas mayores o que la necesiten.
- Crear un módulo de gestión de datos con operaciones CRUD.
- Diseñar una interfaz responsive e intuitiva.
- Aplicar buenas prácticas de programación y seguridad.
- Documentar el proceso de desarrollo y las pruebas realizadas.

2.2 Objetivos técnicos

Más allá de la funcionalidad de negocio, la implementación persigue los siguientes hitos de ingeniería:

- Rendimiento y Adaptabilidad:
 - Server-Side Rendering (SSR): Se ha implementado el renderizado en servidor nativo de Angular 21 para minimizar el *First Contentful Paint (FCP)* y mejorar la indexación (SEO).
 - Optimización del First Contentful Paint (FCP)
 - El sistema prioriza la velocidad de percepción del usuario mediante la optimización de la métrica FCP. En aplicaciones SPA tradicionales, el usuario experimenta una pantalla en blanco hasta que el bundle de JavaScript se descarga y ejecuta. El navegador recibe un documento HTML completo, permitiendo que el FCP ocurra milisegundos después de la solicitud, mejorando drásticamente la experiencia de usuario (UX) y el posicionamiento SEO, ya que los motores de búsqueda reciben contenido indexable de inmediato.
 - Gestión de Entornos Isomórficos: Se ha configurado el servidor Express para manejar las diferencias entre el entorno de ejecución del servidor (Node.js) y

el cliente (Browser). Específicamente, se ha implementado una estrategia de anulación de WebSockets en el lado del servidor para garantizar la compatibilidad de la librería cliente de Supabase durante la fase de prerenderizado, asegurando que la hidratación en el cliente ocurra sin bloqueos ni errores de conexión.

- **Integridad:** Garantizar la consistencia de datos mediante restricciones a nivel de base de datos y validación de esquemas en tiempo de ejecución (Zod).
- **Escalabilidad:** Implementación de un flujo de CI/CD (Integración y Despliegue Continuo) que permite actualizaciones automáticas sin tiempo de inactividad.
- **Seguridad:** Delegación de la autorización al motor de base de datos mediante políticas RLS (*Row Level Security*), asegurando que el aislamiento de datos sea inviolable desde el frontend.

2.3. Alcance

Para acotar el desarrollo de la aplicación a los plazos y recursos disponibles para un Trabajo de Fin de Grado, se han definido claramente las fronteras del sistema, especificando qué funcionalidades se incluyen en la versión final y cuáles quedan descartadas o pospuestas para futuras iteraciones.

2.1. Funcionalidades INCLUIDAS (Dentro del alcance)

El sistema actual es completamente funcional y abarca los siguientes módulos:

- **Gestión de Identidad y Roles:** Sistema de registro y autenticación seguro (JWT) con tres perfiles diferenciados: Administrador, Empresa (Profesionales) y Cliente (Particular).
- **Catálogo de Servicios:** Funcionalidad CRUD (Crear, Leer, Actualizar, Borrar) para que las empresas publiquen y gestionen su oferta de servicios asistenciales.
- **Motor de Contratación:** Flujo completo que permite a los clientes buscar empresas, seleccionar servicios, elegir horarios y formalizar contratos (con opción a cancelación).

- **Comunicación Interna:** Sistema de mensajería (chat) en tiempo real entre empresas y clientes para acordar detalles previos y posteriores a la contratación.
- **Asistencia Inteligente:** Integración de la API de Google Gemini para proporcionar un chatbot de soporte 24/7 entrenado con los manuales de la plataforma.
- **Panel de Administración y Notificaciones:** Dashboard con métricas clave, exportación de documentos (PDF) y un sistema de alertas al administrador vinculado a la API de Telegram.

2.2. Funcionalidades EXCLUIDAS (Fuera del alcance)

A continuación, se detallan los módulos que **no** forman parte del sistema actual, junto con la justificación técnica o de negocio de su exclusión:

- **Pasarela de Pago Integrada (Stripe / PayPal):**
 - ¿Por qué se excluye? La integración de pagos requiere cumplir con normativas estrictas de seguridad financiera (como PCI-DSS) y gestionar facturación real, devoluciones e impuestos. Para el contexto de un TFG, la plataforma CuidaDos actúa puramente como un **conector o intermediario**. Según el protocolo de la aplicación, el acuerdo económico y el pago final se realizan externamente entre el cliente y la empresa una vez formalizado el contacto en la web.
- **Geolocalización avanzada (Mapas interactivos tipo Google Maps):**
 - ¿Por qué se excluye? La integración de mapas de terceros añade dependencias de APIs de pago (que pueden generar costes imprevistos) y complejidad técnica en el frontend. La ubicación de los servicios se gestiona actualmente mediante texto estructurado (ciudad, código postal) que es suficiente para el buscador actual. Queda planteado como una mejora futura (usando PostGIS).
- **Sistema de Videollamadas Nativas:**

- ¿Por qué se excluye? Desarrollar comunicaciones de vídeo en tiempo real requiere servidores especializados (tecnología WebRTC) y un gran ancho de banda, lo cual excede los recursos de un proyecto académico gratuito alojado en Vercel y Supabase. El chat de texto interno se considera suficiente para la comunicación inicial.

3. REQUISITOS

3.1 Requisitos Funcionales

Para usuarios (usuarios que contratan servicios).

Código	Requisito	A quién afecta	Descripción
RF-C01	Registro	Usuarios	Permite a cualquier persona crear una cuenta mediante un formulario, pasando a ser cliente de la plataforma.
RF-C02	Inicio de sesión	Usuarios	Los clientes pueden iniciar sesión usando correo electrónico y contraseña.
RF-C03	Búsqueda de servicios	Usuarios	Los clientes pueden buscar servicios ofrecidos por empresas usando filtros (categoría, ubicación, precio, etc.).
RF-C04	Contratar servicio	Usuarios	Permite contratar un servicio según la disponibilidad ofrecida por la empresa.
RF-C05	Baja de usuario	Usuarios	Permite a los clientes eliminar su cuenta de la plataforma, eliminando todos los contratos.
RF-C06	Calendario personal	Usuarios	Cada cliente dispone de un calendario donde se muestran los servicios contratados.
RF-C07	Notificaciones	Usuarios	Los clientes recibirán notificaciones por correo o dentro de la app al contratar un servicio o ante cambios en los mismos.

RF-C08	Mensajería	Usuarios	Los clientes podrán enviar y recibir mensajes de empresas y administradores.
RF-C09	Cancelar contrato	Usuarios	Permite cancelar un contrato.
RF-C10	Modificar perfil	Usuarios	Los clientes pueden actualizar su información personal desde su perfil.

Tabla 1: RF Usuarios.

Para empresas.

Código	Requisito	A quién afecta	Descripción
RF-E01	Registro	Empresas	Permite a empresas registrarse como proveedoras de servicios.
RF-E02	Iniciar sesión	Empresas	Las empresas pueden iniciar sesión usando correo electrónico y contraseña.
RF-E03	Modificar perfil	Empresas	Las empresas pueden editar su información desde el perfil empresarial.
RF-E04	Calendario empresarial	Empresas	Cada empresa dispone de un calendario donde se muestran servicios programados.
RF-E05	Mensajería	Empresas	Las empresas pueden enviar y recibir mensajes con clientes.
RF-E06	Baja de empresa	Empresas	Permite a empresas eliminar su cuenta de la plataforma.
RF-E07	Modificar servicio-horario	Empresas	Las empresas pueden modificar los servicios-horarios que ofrecen.

RF-E08	Cancelar servicio-horario	Empresas	Las empresas pueden eliminar los servicios-horarios que ofrecen.
RF-E9	Añadir servicio-horario	Empresas	Las empresas pueden añadir servicios-horarios para ofrecerlos.
RF-E10	Notificaciones	Empresas	Las empresas recibirán una notificación ante cualquier cambio por parte del cliente
			en los servicios contratados/ creación de un nuevo contrato.
RF-11	Cancelar un contrato	Empresas	Las empresas pueden cancelar un contrato, siendo esta cancelación notificada al cliente.

Tabla 2: RF Empresas.

Para administradores.

Código	Requisito	A quién afecta	Descripción
RF-A01	Registro	Administradores	Solo un administrador puede dar de alta a otro administrador.
RF-A02	Iniciar sesión	Administradores	Los administradores pueden iniciar sesión usando correo electrónico y contraseña.
RF-A03	CRUD clientes	Administradores	Permite crear, leer, actualizar y eliminar clientes y sus datos.

RF-A04	CRUD empresas	Administradores	Permite crear, leer, actualizar y eliminar empresas y sus datos.
RF-A05	Mensajes	Administradores	Administradores pueden enviar y recibir mensajes de empresas y clientes.
RF-A06	Dashboard	Administradores	Permite consultar métricas y análisis de uso de la plataforma.
RF-A07	Notificaciones	Administradores	Los administradores recibirán una notificación ante cualquier incidencia en los servicios, creada por los usuarios.
RF-A8	Baja	Administradores	Los administradores pueden darse de baja.
RF-A9	Modificar perfil	Administradores	Los administradores pueden editar su información desde el perfil.
RF-A10	CRUD Servicios	Administradores	Los administradores pueden añadir, eliminar o crear servicios.
RF-A11	CRUD Horarios	Administradores	Los administradores pueden añadir, eliminar o crear horarios.

Tabla 3: RF Administradores.

3.2 Requisitos no Funcionales

Código	Requisito	A quién afecta	Descripción
RNF-01	Interfaz sencilla e intuitiva	Todos	UI/UX clara y accesible para facilitar la navegación a usuarios, empresas y administradores.

RNF-02	Cifrado de contraseñas	Todos	Uso de hashing seguro (ej. bcrypt) para proteger credenciales almacenadas.
RNF-03	Protección de datos	Todos	Cumplimiento de la normativa vigente de protección de datos (RGPD), asegurando privacidad y control de datos.
RNF-04	Autenticación	Usuarios y empresas	Solo usuarios autenticados pueden acceder a funcionalidades privadas (contratación de servicios, historial, mensajes, notificaciones...).
RNF-05	Adaptabilidad / Responsive design	Todos	Acceso desde móvil, tablet u ordenador, adaptando la interfaz a distintos tamaños de pantalla.
RNF-06	Multilenguaje	Todos	Plataforma disponible en inglés y español, con posibilidad de
			agregar otros idiomas en el futuro.
RNF-09	Escalabilidad	Todos	El sistema debe poder crecer para soportar más usuarios y empresas sin degradar rendimiento.
RNF-10	Seguridad de la API	Todos	Uso de HTTPS, autenticación JWT y validaciones estrictas para prevenir accesos no autorizados.

RNF-11	Compatibilidad de navegadores	Todos	Funcionamiento correcto en Chrome, Edge, Firefox, Safari y versiones recientes de cada uno.
RNF-12	Mantenimiento	Administradores	Permitir actualizaciones sin interrumpir gravemente el servicio.
RNF-13	Compatibilidad de dispositivos	Todos	Funcionamiento correcto para todo tipo de dispositivos.
RNF-14	Requisitos de software	Todos	Cualquier sistema operativo. Entorno de desarrollo: VSCode. Lenguaje de programación: Typescript. Base de datos: PostgreSQL

Tabla 4: RNF.

3.3 Requisitos para el Cliente (Usuario Final)

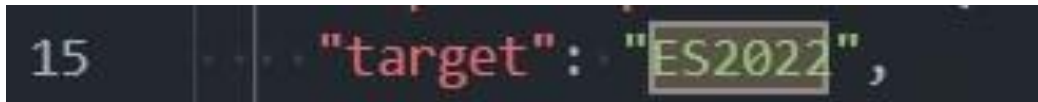
Para garantizar la correcta interpretación del código compilado y la estabilidad de la conexión en tiempo real, se establecen los siguientes requisitos:

1. Navegador Compatible con ESModules (ES2022): La aplicación ha sido compilada bajo el estándar ECMAScript 2022 para aprovechar las últimas optimizaciones de rendimiento de Angular 21. Se requiere soporte nativo para módulos de JavaScript.
 - *Google Chrome*: Versión 90 o superior.
 - *Mozilla Firefox*: Versión 90 o superior.
 - *Safari*: Versión 15 o superior.
 - *Microsoft Edge*: (Chromium based) Versión 90 o superior.

- **Nota:** Internet Explorer 11 no está soportado.

Motivo:

En el archivo tsconfig.json se ha configurado:



```
15  ...  "target": "ES2022",
```

Ilustración 1: tsconfig.json

Angular está compilando el código a una versión muy moderna.

Si se usa un **navegador viejo** (ej. Chrome 80): La aplicación se romperá y saldrá una pantalla blanca porque el navegador no entenderá instrucciones modernas.

ESModules: Angular 21 ya no genera scripts "clásicos", genera Módulos. Solo los navegadores modernos (Chrome 90+, etc.) saben cargar estos módulos de forma nativa.

- **Conectividad de Red (WebSockets):** Además del tráfico HTTP estándar (puerto 443), la red del usuario debe permitir conexiones WebSocket (WSS) salientes hacia el dominio de Supabase. El sistema implementa una arquitectura de Sincronización de Estado en Tiempo Real (Realtime Database), que, a diferencia de las aplicaciones web tradicionales que requieren recargar la página para ver cambios, **cuidaDos** mantiene una **conexión bidireccional** abierta para escuchar eventos INSERT, UPDATE y DELETE directamente desde el motor de base de datos (PostgreSQL WAL). Crítico para los siguientes módulos:
 1. **Integridad de Catálogo (Servicios y Horarios):** Si una Empresa elimina un hueco horario o modifica una tarifa, el cambio se refleja instantáneamente en la interfaz de los Clientes que están visualizando la oferta en ese momento, evitando errores de contratación por información obsoleta (*Race Conditions*).
 2. **Gestión Administrativa (Usuarios y Empresas):** Los paneles CRUD del administrador se actualizan en vivo. Si se registra una nueva empresa o se da

de baja un usuario, la tabla de datos se refresca automáticamente sin intervención humana.

3. Gestión de los servicios, horarios y ofertas ofrecidas por las empresas: los administradores pueden gestionar los servicios y horarios que se ofrecen en tiempo real, así como visualizarlos y modificarlos en una tabla de gestión. Las empresas pueden gestionar (crear, ver, eliminar, modificar) las ofertas que ofrecen en tiempo real a su vez.

4. Mensajería y Notificaciones: Recepción instantánea de mensajes y actualizaciones de estado (leído/no leído).

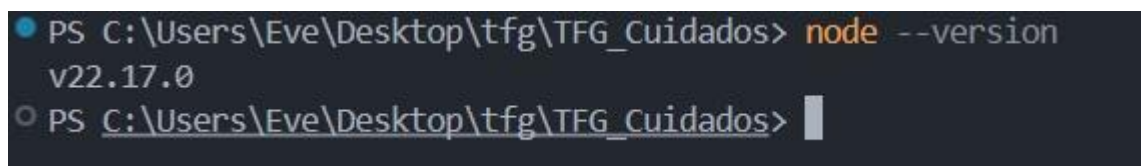
Beneficio: Esta estrategia elimina la necesidad de realizar *Polling* (peticiones HTTP repetitivas cada X segundos para consultar cambios), reduciendo drásticamente el consumo de ancho de banda y la carga computacional tanto en el cliente como en el servidor.

3.4 Requisitos² para el Servidor de Desarrollo

Para transformar el código fuente (TypeScript, SASS) en una aplicación interpretable por los navegadores, se requiere un entorno de construcción local basado en las siguientes herramientas:

- Node.js (Runtime Environment)

Versión: v18.13.0 (LTS) o superior. El proyecto cuenta actualmente con la versión: v22.17.0



```
PS C:\Users\Eve\Desktop\tfg\TFG_Cuidados> node --version
v22.17.0
PS C:\Users\Eve\Desktop\tfg\TFG_Cuidados>
```

Ilustración 2. Node version

² Los comandos correspondientes para las instalaciones se encuentran en el apartado [5.1 Instalación de Prerrequisitos](#).

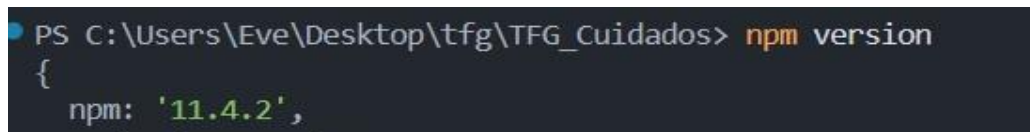
Aunque **cuidaDos** no utiliza un servidor Node.js en producción (ya que el backend es Supabase), este entorno es indispensable durante la fase de desarrollo. Actúa como el motor de ejecución que permite al compilador de Angular (Esbuild/Webpack) transpilar el código TypeScript a JavaScript (ECMAScript) y gestionar el renderizado del lado del servidor (SSR) en local.

- NPM (Node Package Manager)

Versión: v10.0.0 o superior.

Es el gestor de dependencias encargado de resolver e instalar las librerías listadas en el archivo package.json (como @angular/material, zod, supabase-js). Garantiza que todo el equipo de desarrollo y el servidor de despliegue (Vercel) utilicen exactamente las mismas versiones de cada paquete mediante el archivo de bloqueo package-lock.json.

El proyecto actualmente cuenta con la versión: 11.4.2



```
PS C:\Users\Eve\Desktop\tfg\TFG_Cuidados> npm version
{
  npm: '11.4.2',
```

Ilustración 3: NPM versión.

Listado de dependencias:

Comando → npm list --depth=0


```
PS C:\Users\Eve\Desktop\tfg\TFG_Cuidados> npm list --depth=0
Eve@ C:\Users\Eve
├── @angular/cdk@21.0.5
├── @angular/material@21.0.5
├── @emailjs/browser@4.4.1
├── @ngx-translate/core@17.0.0
├── @ngx-translate/http-loader@17.0.0
├── @supabase/supabase-js@2.90.1
├── @tailwindcss/postcss@4.1.18
├── @tailwindcss/vite@4.1.18
├── autoprefixer@10.4.23
├── chart.js@4.5.1
├── lucide-angular@0.562.0
├── ng2-charts@8.0.0
├── patch-package@8.0.1
├── postcss@8.5.6
└── tailwindcss@4.1.18
```

Ilustración 4: Lista de dependencias.

- Git (Control de Versiones)

Herramienta fundamental para la Integración Continua (CI).

Historial: Permite un control granular de los cambios en el código.

Despliegue Automático: Vercel está conectado al repositorio remoto. Al detectar un git push en la rama main, Vercel descarga el código y ejecuta el script de construcción automáticamente. Sin Git, el flujo CI/CD no sería posible.

Ejemplo de uso de git:

```
PS C:\Users\Eve\Desktop\tfg\TFG_Cuidados> git status
On branch fix_tests
Your branch is up to date with 'origin/fix_tests'.

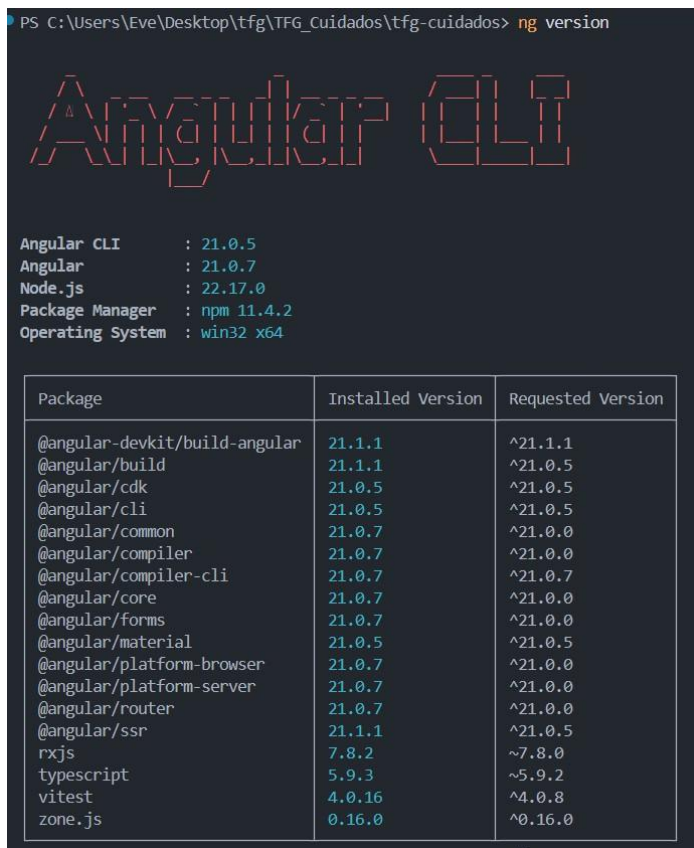
nothing to commit, working tree clean
```

Ilustración 5: Git implementado.

Git status demuestra que no hay cambios que realizar.

- Angular CLI (Command Line Interface)

```
PS C:\Users\Eve\Desktop\tfg\TFG_Cuidados\tfg-cuidados> ng version
```



Package	Installed Version	Requested Version
@angular-devkit/build-angular	21.1.1	^21.1.1
@angular/build	21.1.1	^21.0.5
@angular/cdk	21.0.5	^21.0.5
@angular/cli	21.0.5	^21.0.5
@angular/common	21.0.7	^21.0.0
@angular/compiler	21.0.7	^21.0.0
@angular/compiler-cli	21.0.7	^21.0.7
@angular/core	21.0.7	^21.0.0
@angular/forms	21.0.7	^21.0.0
@angular/material	21.0.5	^21.0.5
@angular/platform-browser	21.0.7	^21.0.0
@angular/platform-server	21.0.7	^21.0.0
@angular/router	21.0.7	^21.0.0
@angular/ssr	21.1.1	^21.0.5
rxjs	7.8.2	~7.8.0
typescript	5.9.3	~5.9.2
vitest	4.0.16	^4.0.8
zone.js	0.16.0	^0.16.0

Ilustración 6: Angular cli.

Versión: v21.0.5.

Es la herramienta de ingeniería central del proyecto. Actúa como una capa de abstracción sobre el sistema de construcción (basado en *Esbuid* en las versiones modernas), gestionando el ciclo de vida completo del desarrollo de software sin necesidad de configurar manualmente herramientas complejas como Webpack

Es la herramienta de orquestación del framework. Abstrae la complejidad de la configuración de Webpack y permite:

- **Scaffolding (Generación de Estructura):** Automatiza la creación de artefactos de código (Componentes, Servicios, Interfaces) mediante el comando `ng generate`. Garantiza que todos los archivos sigan estrictamente la **Guía de Estilo oficial de Angular**, creando automáticamente los archivos de pruebas unitarias (`.spec.ts`) y actualizando las importaciones necesarias en los componentes *Standalone*.

- **Serving (Servidor de Desarrollo):** Despliega un servidor HTTP local en memoria a través del comando `ng serve`.
 - **Característica Clave:** Implementa **HMR (Hot Module Replacement)**. Esta tecnología detecta cambios en el código TypeScript o CSS y actualiza el módulo específico en el navegador sin recargar la página completa, manteniendo el estado de la aplicación (como los datos de un formulario relleno) durante el desarrollo.
- **Building (Compilación para Producción):** Ejecuta el pipeline de optimización mediante `ng build`. Este proceso transforma el código legible por humanos en código máquina eficiente para el navegador:
 - **Transpilación:** Convierte TypeScript a JavaScript (ECMAScript 2022).
 - **AOT (Ahead-of-Time):** Compila las plantillas HTML y CSS en instrucciones JavaScript durante la construcción, eliminando la necesidad de enviar el compilador de Angular al navegador.
 - **Tree-Shaking:** Elimina el código muerto (funciones o librerías importadas pero no utilizadas) para reducir el peso final.
 - **Hashing:** Añade códigos únicos a los nombres de archivo (ej: `main.7a2b3c.js`) para gestionar la caché del navegador en futuras actualizaciones.

4. HERRAMIENTAS Y STACK TECNOLÓGICO

La arquitectura de `cuidaDos` se ha diseñado siguiendo el principio de Separation of Concerns (SoC), seleccionando librerías especializadas para cada capa de la aplicación. A continuación, se detalla la configuración técnica, la justificación de uso y los comandos de integración de cada componente.

4.1 Frontend: Core y Framework

Angular 21 (v21.0.5)

Framework principal del proyecto. Se ha configurado utilizando la arquitectura de Standalone Components, eliminando la complejidad de los NgModules tradicionales y reduciendo el *boilerplate*.

- Innovación (Signals & SSR):
 - o Signals API: Se utiliza como primitiva reactiva principal para la gestión del estado, sustituyendo el uso intensivo de Zone.js. Esto permite una detección de cambios granular (solo se actualiza en el DOM lo que realmente cambia), mejorando el rendimiento en dispositivos de gama baja.
 - o Hydration: Configurado con `@angular/ssr` para permitir una "hidratación no destructiva". El servidor envía el HTML pre-renderizado y Angular "despierta" la página sin parpadeos ni reconstrucción del DOM.
- Comando de Creación: El proyecto se inicializó habilitando SSR y el enrutamiento estricto:

```
ng new tfg-cuidados --ssr --style=css --routing
```

Verificación de la versión del framework (v21.0.5) y entorno Node.js.:

```
ng version
```

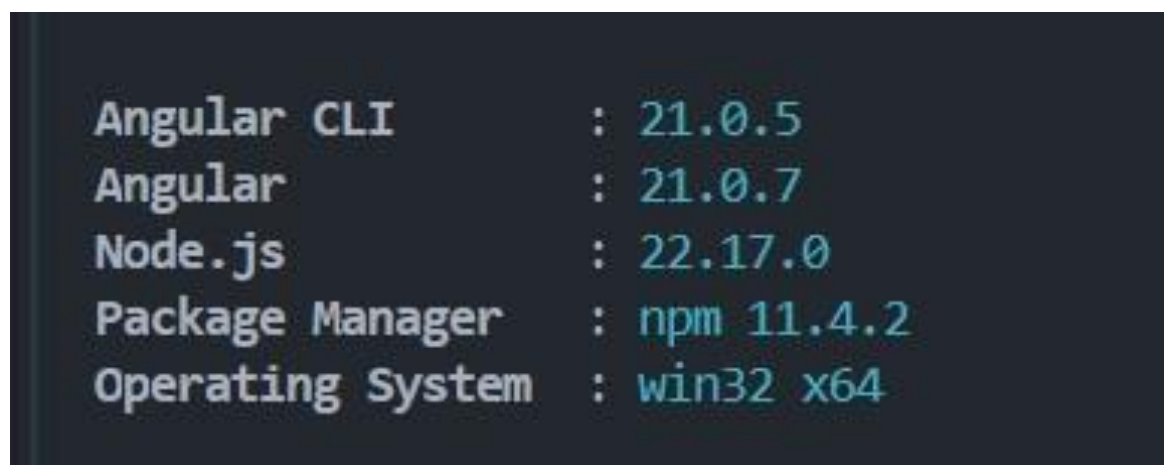


Ilustración 7: Versión de Angular.

4.2 Estilos e Interfaz de Usuario (UI)

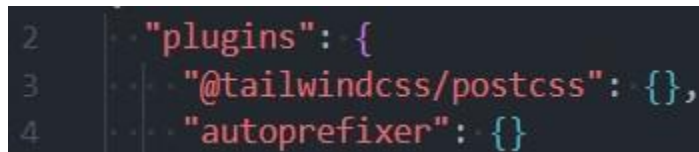
4.2.1 Tailwind CSS (Motor de Estilos)

Framework *Utility-First*. A diferencia de las hojas de estilo tradicionales, Tailwind permite construir diseños personalizados directamente en el HTML. Se ha integrado mediante **PostCSS** (como se evidencia en el archivo `.postcssrc.json`), lo que permite purgar el CSS no utilizado (*PurgeCSS*) en tiempo de compilación, generando archivos de estilo extremadamente ligeros.

Comando de Instalación:

```
npm install -D tailwindcss postcss autoprefixer npx  
tailwindcss init
```

Configuración del preprocesador PostCSS para la integración nativa de Tailwind:



```
2 'plugins': {  
3   '@tailwindcss/postcss': {},  
4   'autoprefixer': {}
```

Ilustración 8: Archivo de Tailwind.

4.2.2 Angular Material (Componentes UI)

Biblioteca oficial utilizada para componentes complejos que requieren cumplir estrictamente con los estándares de accesibilidad (WCAG). Se emplea fundamentalmente en:

- MatDialog: Para ventanas modales de confirmación y formularios emergentes.
- MatTable: Para la visualización tabular de datos con paginación y ordenamiento en el panel de administración.

Comando de Instalación: ng

```
add @angular/material
```

4.2.3 Lucide Angular (Iconografía)

Set de iconos SVG optimizados. Se seleccionó frente a FontAwesome por su capacidad de **Tree-Shaking**: el compilador de Angular elimina automáticamente del *bundle* final

cualquier icono que no se esté importando explícitamente, reduciendo el peso de la aplicación.

Comando de Instalación: npm

install lucide-angular

4.2.4 Modos oscuro

Implementación del Modo Oscuro (Dark Mode)

Para mejorar la accesibilidad y reducir la fatiga visual de los usuarios, se ha implementado un sistema de temas dinámico. Se ha utilizado la estrategia darkMode: 'class' de **Tailwind CSS**, permitiendo un control granular de los estilos.

A nivel de arquitectura de Angular, la gestión del estado se realiza mediante un servicio global (ThemeService) que utiliza **Signals** (signal<boolean>) para una reactividad óptima. Este servicio interactúa directamente con el DOM (document.documentElement.classList) y persiste la preferencia del usuario en el localStorage del navegador.

Además, se escucha la configuración del sistema operativo del usuario mediante window.matchMedia('(prefers-color-scheme: dark)') para aplicar el tema adecuado desde el primer renderizado.

4.2.5 Responsive

Para el diseño de la aplicación responsive, se usó la aplicación «responsively app», disponible de forma gratuita en: <https://responsively.app/>

Con el uso de esta aplicación se diseñó y probó la aplicación, comprobando que esta fuera responsive.

4.3 Backend & Persistencia (BaaS)

Supabase Client (@supabase/supabase-js)

SDK isomórfico que permite la comunicación con la infraestructura de Supabase.

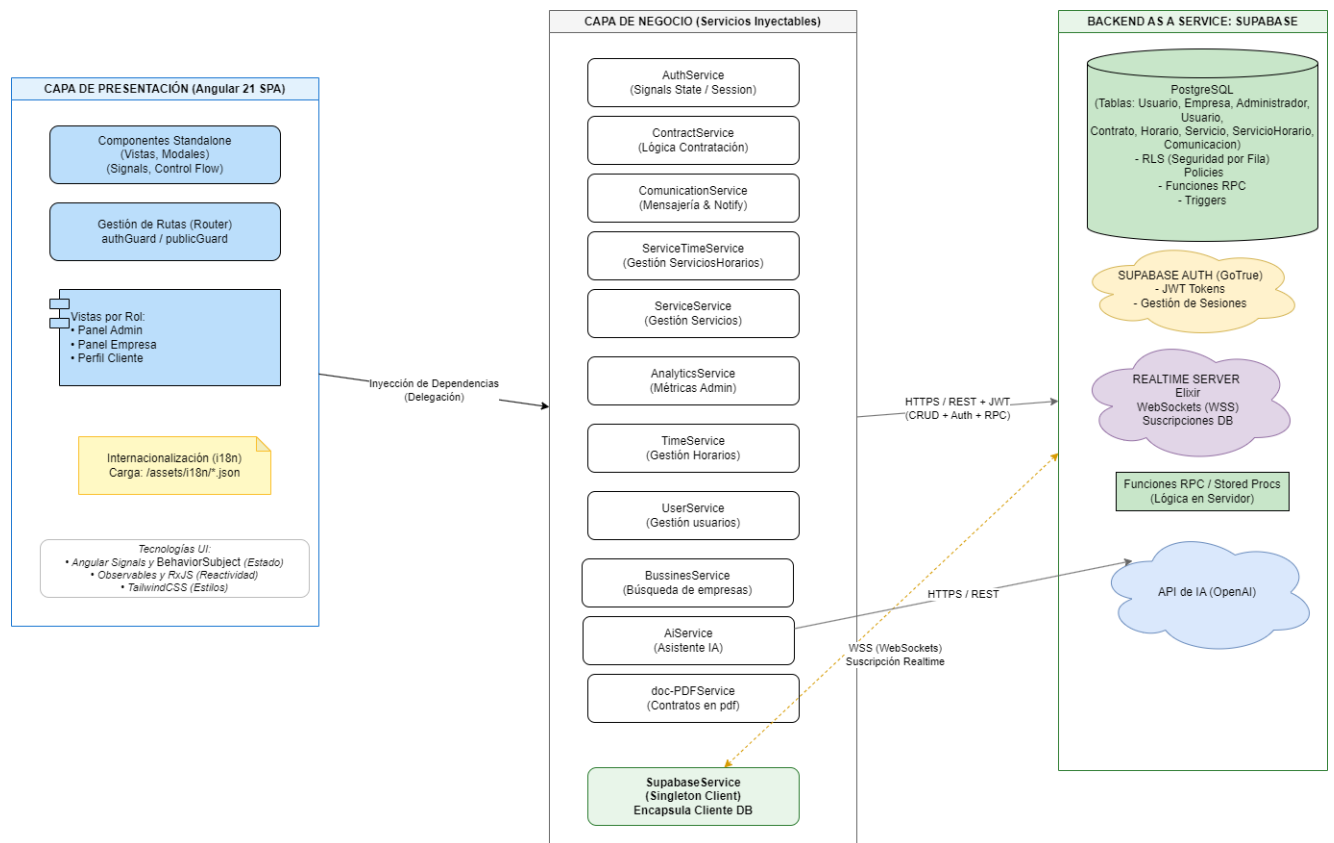
- **Funcionalidad:** Gestiona la autenticación de usuarios (GoTrue), las consultas a la base de datos PostgreSQL (PostgREST) y mantiene los canales de WebSocket abiertos para la funcionalidad *Realtime* (chat y actualizaciones en vivo).
- **Integración:** Se ha implementado mediante un servicio inyectable (SupabaseService) que sigue el patrón **Singleton**, garantizando una única instancia de conexión durante el ciclo de vida de la sesión.

Comando de Instalación: npm

install @supabase/supabase-js

4.4 Librerías Auxiliares

- **Zod:** Validación estricta de esquemas TypeScript. Se usa para validar formularios antes de enviarlos a la API.:
 - ❖ **npm:** npm install zod
 - ❖ **yarn:** yarn add zod
- **EmailJS:** Servicio de notificaciones transaccionales.
 - ❖ **npm:** npm install @emailjs/browser
 - ❖ **yarn:** yarn add @emailjs/browser
- **Chart.js / ng2-charts:** Visualización de datos en el panel de administración.
 - ❖ **npm:** npm install chart.js ng2-charts
 - ❖ **yarn:** yarn add chart.js ng2-charts



Se trata de un modelo moderno y desacoplado, diseñado bajo el patrón de separación de responsabilidades (Separation of Concerns).

El sistema prescinde de un servidor backend tradicional (como Node.js o Java) y se comunica directamente desde el cliente con un servicio BaaS (*Backend as a Service*).

La arquitectura se divide en tres bloques o capas principales. A continuación, te detallo cada una:

1. CAPA DE PRESENTACIÓN (Angular 21 SPA)

Ubicada a la izquierda, representa todo lo que ocurre en el navegador del usuario (Frontend).

- **Componentes Standalone:** La interfaz está construida con componentes independientes (vistas y modales) utilizando las últimas características de Angular como *Signals* y *Control Flow* para un rendimiento óptimo.
- **Vistas por Rol:** La interfaz cambia dinámicamente ofreciendo un Panel de Administrador, un Panel de Empresa o un Perfil de Cliente, dependiendo de quién inicie sesión.

- **Gestión de Rutas y Seguridad:** El enrutador (Router) utiliza los guardianes `authGuard` y `publicGuard` para proteger las páginas y evitar que usuarios sin sesión accedan a zonas privadas.
- **Internacionalización (i18n):** Carga archivos JSON de forma dinámica para traducir la aplicación a diferentes idiomas.
- **Tecnologías UI:** Se apoya en *Signals* y *RxJS/BehaviorSubjects* para reaccionar a los cambios de datos en tiempo real, y utiliza **TailwindCSS** para todo el diseño visual.

2. CAPA DE NEGOCIO (Servicios Inyectables)

Ubicada en el centro, actúa como el "cerebro" o intermediario. Para no saturar la vista con código complejo, los componentes de Angular *delegan* la lógica a estos servicios mediante **Inyección de Dependencias**.

- **Servicios de Dominio:** Vemos una lista de servicios especializados, cada uno con una responsabilidad única (ej. `AuthService` para sesiones, `ContractService` para contrataciones, `AnalyticsService` para las gráficas del administrador, `doc-PDFService` para generar documentos, etc.).
- **SupabaseService (Singleton Client):** Es el servicio más importante de conexión. Se instancia una única vez (Singleton) y encapsula el cliente oficial de Supabase, centralizando todas las llamadas a la base de datos para que el resto de los servicios no tengan que configurarlas desde cero.

3. BACKEND AS A SERVICE (Supabase y APIs Externas)

Ubicada a la derecha, representa la infraestructura en la nube.

- **PostgreSQL:** Es el corazón del almacenamiento. Contiene todas las tablas (Usuario, Contrato, Servicio, etc.). La seguridad está garantizada directamente aquí mediante **RLS (Row Level Security)**, lo que impide que un usuario vea datos de otros. También alberga **Triggers** y **Funciones RPC** (procedimientos almacenados) que ejecutan lógica de negocio crítica en el servidor.

- **Supabase Auth (GoTrue):** Es el módulo que expide los **tokens JWT** y gestiona el inicio de sesión.
- **Realtime Server (Elixir):** Permite mantener conexiones vivas a través de **WebSockets (WSS)**. Esto es lo que hace que el chat o las notificaciones lleguen al instante sin tener que recargar la página.
- **API de IA Externa:** El diagrama muestra cómo el AiService (de la capa intermedia) se conecta de forma asíncrona mediante HTTPS/REST a una API de Inteligencia artificial para alimentar el chatbot del aplicativo (en tu manual mencionas que usaste Google Gemini, el diagrama ilustra esa misma conexión conceptual).

Flujo de Comunicación (Las flechas)

El flujo de datos va de izquierda a derecha:

1. El usuario hace clic en la vista (Capa 1).
2. La vista llama a un servicio (Capa 2).
3. El servicio lanza una petición **HTTPS/REST** adjuntando el Token JWT (para demostrar quién es) hacia la base de datos, o establece un túnel **WSS (WebSocket)** para escuchar cambios en vivo (Capa 3).

Gestión de usuarios y autenticación

Lógica de Autenticación (Backend - Supabase Auth)

La lógica de autenticación de **CuidaDos** no reside en un servidor tradicional propio, sino que se delega en **Supabase Auth**, que actúa como el proveedor de identidad (IdP).

- **Protocolo:** Se utiliza el estándar **JWT (JSON Web Tokens)**. Cuando un usuario inicia sesión, Supabase genera un token firmado que contiene la identidad del usuario y sus metadatos.
- **Esquema de Seguridad:** La autenticación está totalmente aislada de la lógica de negocio. Supabase gestiona las credenciales (email y contraseña cifrada) en un esquema interno

protegido llamado `auth`, mientras que los datos del perfil se sincronizan con la tabla `public.Usuario` mediante un *Trigger* automático (`handle_new_user`).

- **Validación:** Cada petición realizada desde el cliente hacia la base de datos incluye el JWT en las cabeceras. PostgreSQL valida este token de forma nativa para aplicar las políticas de **RLS (Row Level Security)**, permitiendo o denegando el acceso a los registros en tiempo real sin necesidad de un backend intermediario.

Manejo de la Sesión en el Frontend (Angular)

En la capa de aplicación, la gestión del estado de autenticación se centraliza en un servicio dedicado (`AuthService`) utilizando las siguientes estrategias:

- **Persistencia de Tokens:** El cliente de Supabase (`@supabase/supabase-js`) gestiona automáticamente el almacenamiento del JWT y el *refresh token* en el **LocalStorage** del navegador. Esto permite que la sesión sea persistente incluso si el usuario recarga la página.
- **Gestión de Estado:** Se utilizan **Angular Signals** (o `BehaviorSubjects`) para difundir de forma reactiva el estado del usuario (conectado/desconectado) a todos los componentes de la interfaz, permitiendo cambios dinámicos en la barra de navegación y los menús.
- **Interceptores:** Aunque Supabase maneja gran parte de la comunicación, para otras APIs externas (como el Asistente de IA o Telegram), el sistema está preparado para inyectar las claves necesarias desde las variables de entorno de forma segura.

Protección de Rutas (Guards)

Para garantizar que solo los usuarios autorizados accedan a secciones sensibles (como el Panel de Administración o el Perfil de Empresa), se han implementado **Angular Guards**:

- **authGuard:** Verifica la existencia de una sesión activa y válida antes de permitir el acceso a rutas privadas. En caso de no existir un token válido, redirige al usuario automáticamente a la página de inicio de sesión.
- **publicGuard:** Evita que usuarios ya autenticados accedan a páginas innecesarias, como el formulario de registro o login, redirigiéndolos directamente a su panel principal según su rol.

4.5 Testing

El proyecto implementa una estrategia de pruebas en pirámide, asegurando que cada componente, desde la lógica interna hasta la interfaz de usuario y la comunicación con el servidor, funcione según los requerimientos.

4.5.1 Karma & Jasmine: Pruebas Unitarias

Las pruebas unitarias se centran en probar la unidad mínima de código (funciones, servicios o componentes aislados) para asegurar que la lógica de negocio sea robusta.

- **Jasmine:** Es el framework de desarrollo guiado por comportamiento (BDD) que proporciona la sintaxis para escribir las pruebas (describe, it, expect).
- **Karma:** Es el ejecutor de pruebas (*Test Runner*) que abre un navegador (Chrome por defecto) para ejecutar los tests y reportar los resultados en tiempo real.

Comandos de Instalación y Uso Normalmente incluido en Angular CLI.

Si necesitas reinstalar:

```
npm install jasmine-core karma karma-jasmine karma-chrome-launcher --save-dev
```

Ejecución de pruebas (Modo Watch):

```
ng test
```

Ejecución para CI: **ng test --watch=false --browsers=ChromeHeadless**

Detalles Técnicos

- **Ubicación:** Archivos con extensión .spec.ts junto al archivo fuente.
- **Aislamiento:** Se utiliza TestBed para crear entornos controlados, simulando dependencias mediante *Mocks* o *Spies* para no depender de servicios externos o APIs reales durante la prueba.

Ej. de página con test para auth:

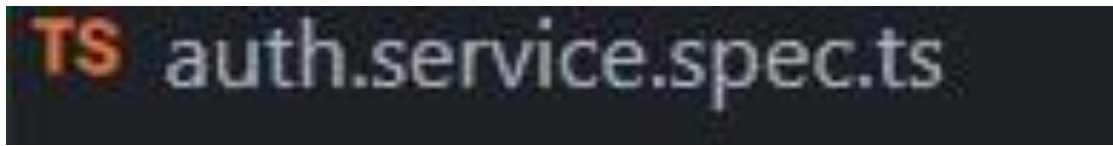


Ilustración 9: Archivo para test Karma.

4.5.1.1 Reportes de karma

Reporte Visual en HTML (El más recomendado) npm

install karma-jasmine-html-reporter --save-dev

Generar el Reporte de Cobertura

Para "descargar" los resultados en un formato profesional (HTML): **ng**

test --watch=false --code-coverage

```
TOTAL: 164 SUCCESS

===== Coverage summary =====
Statements : 64.06% ( 1148/1792 )
Branches   : 50.41% ( 305/605 )
Functions  : 47.98% ( 262/546 )
Lines      : 66.6% ( 1093/1641 )
=====
```

Ilustración 10. Test Karma.

1. Se creará una carpeta llamada /coverage en la raíz del proyecto.
2. Dentro habrá otra carpeta con el nombre del proyecto.
3. Debe abrir el archivo index.html que está dentro.
4. Verá un desglose detallado por líneas de código, funciones y ramas.

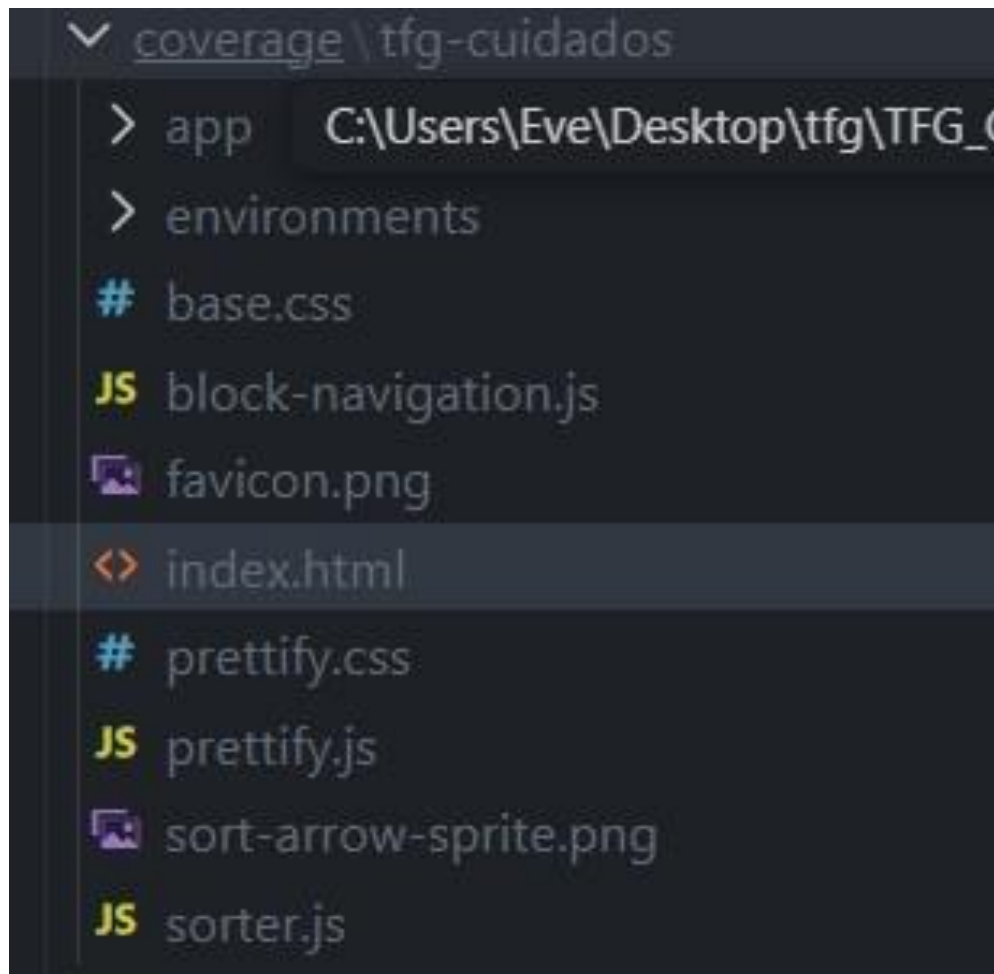


Ilustración 11: Ver test Karma.

All files

64.06% Statements 1148/1792 50.41% Branches 305/605 47.98% Functions 262/546 66.6% Lines 1093/1641

Ilustración 12: Test Karma genéricos.

4.5.2 Cypress: Pruebas de Extremo a Extremo (E2E)

Cypress es una herramienta moderna que simula el comportamiento real de un usuario en el navegador. A diferencia de las unitarias, estas pruebas validan flujos completos.

Comandos de Instalación y Uso

Se recomienda el uso del esquema oficial de Angular para una integración limpia:

Instalación e integración automática:

ng add @cypress/schematic

Abrir la interfaz interactiva de Cypress: **npx**

cypress open

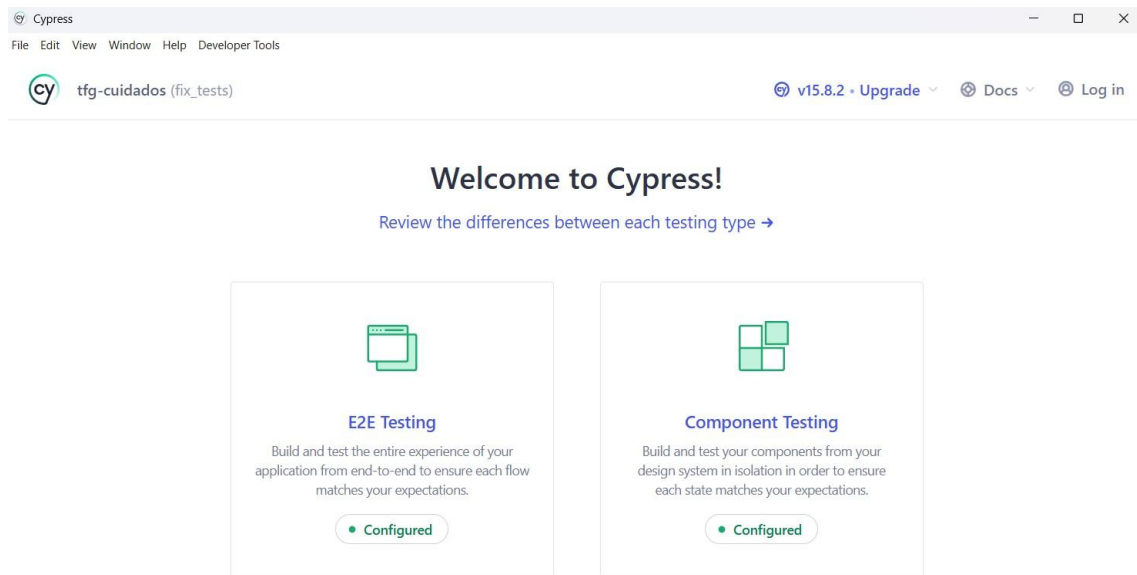


Ilustración 13: Test Cypress.

Ejecutar todas las pruebas en consola (Headless):

npx cypress run

Detalles Técnicos

Simulación Dinámica: Cypress "toma el control" del navegador. Puede hacer clic, escribir en formularios de registro y verificar que los gráficos de **Chart.js** se rendericen correctamente.

Persistencia: Es ideal para probar el flujo de **auto-registro** y la gestión de usuarios por parte del **Administrador**, validando que los permisos de acceso funcionen en la interfaz real.

4.5.3 Postman: Validación de API y Contratos de Datos

Postman es la herramienta de pruebas de capa de red. Se utiliza para garantizar que el intercambio de información entre el frontend (Angular) y el backend sea correcto.

Instalación

Software: Descarga la aplicación de escritorio en postman.com.

VS Code: Extensión oficial "Postman" para integración directa en el editor.



Ilustración 14: Logo Postman.

Detalles Técnicos

Validación de Esquemas: Antes de usar **Zod** en el frontend, se usa Postman para asegurar que el backend devuelve exactamente los campos esperados (evitando errores como el de la columna inexistente rol).

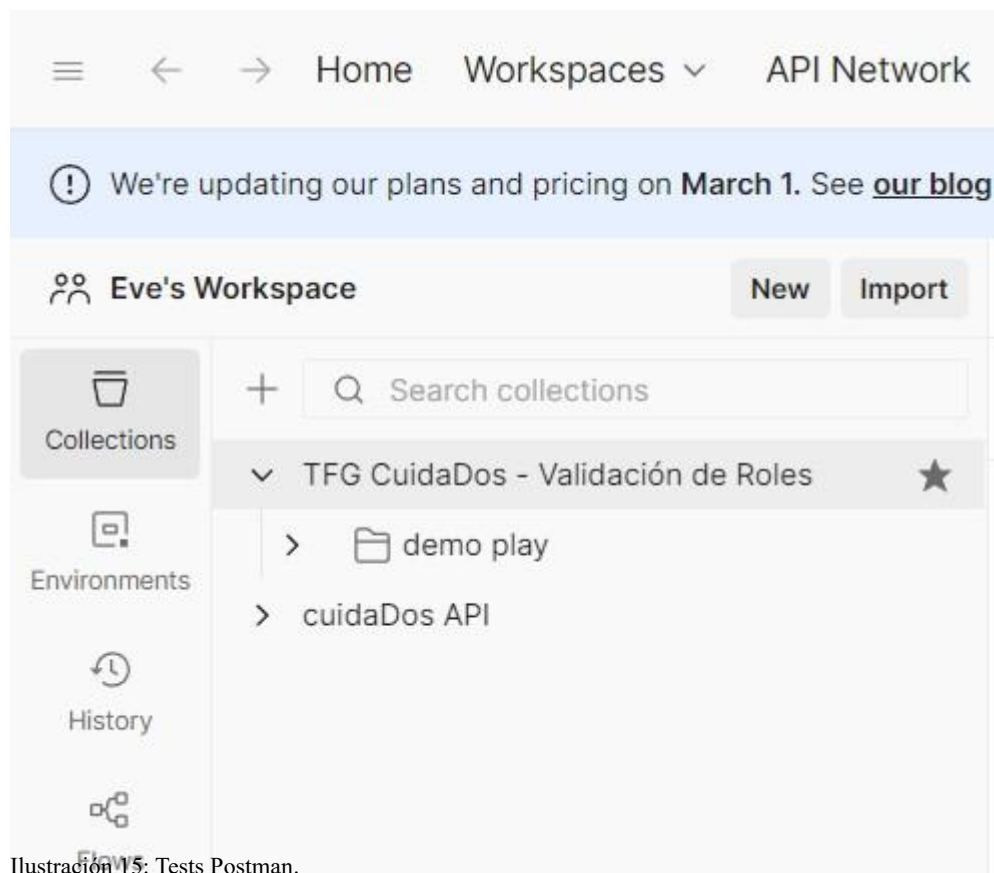


Ilustración 15: Tests Postman.

Automatización: Permite crear "Collections" con scripts de prueba en JavaScript para verificar códigos de estado (200 OK, 201 Created) y tiempos de respuesta.

4.5.3.1 Validación de API con Postman

Debido a que el proyecto utiliza una arquitectura distribuida (Frontend en Angular y Backend en Supabase), se ha implementado una suite de pruebas en **Postman**. Estas pruebas garantizan que la lógica de seguridad y el intercambio de datos sean correctos, complementando los tests unitarios de Karma.

4.5.3.1.1 Configuración del Entorno (Environment)

Para evitar la duplicación de datos y facilitar el cambio entre entornos, se utiliza el archivo `TFG_CuidaDOS.postman_environment.json`.

- **Variables Globales:** Se gestionan mediante variables dinámicas como `{{URL_SUPABASE}}`, `{{ANON_KEY}}` y `{{SERVICE_KEY}}`.

- **Gestión de Tokens:** El entorno almacena automáticamente el `{{token_admin}}` y `{{token_cliente}}` tras el inicio de sesión, permitiendo probar rutas protegidas sin intervención manual.
- **IDs Temporales:** Se utilizan variables como `{{temp_id_servicio}}` o `{{temp_id_usuario}}` para encadenar pruebas (ej: crear un usuario y usar su ID inmediatamente para borrarlo).

4.5.3.1.2 Estructura de la Colección: "Validación de Roles"

La colección principal organiza las peticiones por niveles de acceso, reflejando las reglas de negocio del TFG.

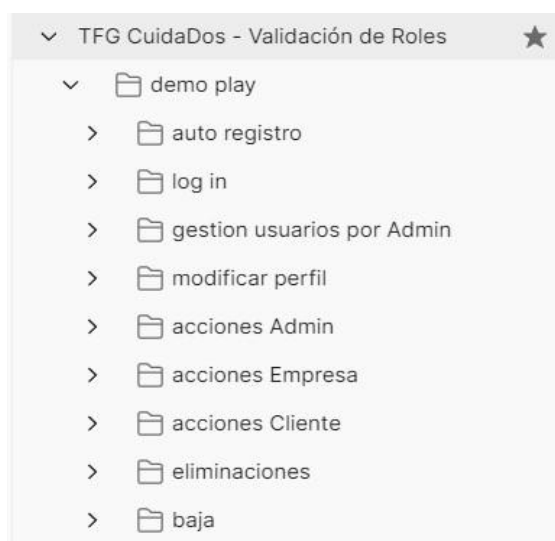


Ilustración 16: Colección test Postman.

1. Auto-registro: tests para el registro individual de cada usuario, uno por cada rol: administrador, empresa y cliente.
2. Log in: tests para el inicio de sesión de cada usuario, por roles.
3. Gestión usuarios por Admin: la gestión (CRUD) de empresas y clientes por parte del administrador de la aplicación.
4. Modificar perfil: modificación de los perfiles de los usuarios, por roles, de forma personal (sin intervención del administrador).
5. Acciones Admin, acciones empresas y acciones clientes: cada una de las funciones por roles.

Administrador: comunicaciones, gestión de servicios y gestión de horarios.

Empresa: gestión de ofertas, comunicaciones.

Cliente: comunicaciones, gestión de contratos.

6. Eliminaciones: eliminaciones lógicas de la base de datos (mensajes, notificaciones...)

7. Baja: baja de la aplicación de los usuarios de prueba.

4.5.3.1.3 Automatización de Pruebas (Scripts)

Cada petición incluye scripts de validación en JavaScript para asegurar que la API responde según lo esperado. Ejemplos incluidos en los archivos:

4.5.3.1.4 Ejecución de Pruebas (Test Run Results)

Según el reporte de ejecución (postman_test_run.json), se han obtenido los siguientes resultados de calidad:

- **Total de Pruebas Ejecutadas:** 94 tests.
- **Tasa de Éxito:** 100% (94 pass / 0 fail).
- **Tiempo de Respuesta Promedio:** ~20ms - 150ms dependiendo del endpoint.

4.6 Integración con Inteligencia Artificial

La integración del asistente conversacional no se limita a una simple llamada a una API; se ha diseñado una arquitectura robusta para garantizar la experiencia de usuario y la seguridad de la información.

La plataforma incorpora un asistente conversacional para resolver dudas de los usuarios en tiempo real, impulsado por el modelo **Gemini 2.0 Flash de Google**. La integración se ha realizado de forma nativa en Angular mediante el módulo HttpClient.

Se ha diseñado un servicio específico (AiService) que actúa como puente con la API REST de Google Generative Language. Se ha aplicado *Prompt Engineering* a través de instrucciones de sistema (systemInstruction) para restringir el contexto del bot exclusivamente a las reglas de negocio de CuidaDos.

Para gestionar la asincronía y el renderizado fluido de las respuestas en el componente visual (AiAssistantComponent), se ha hecho uso de NgZone y ChangeDetectorRef, evitando bloqueos en el hilo principal. Además, se ha implementado un sistema de *fallback* (respaldo manual) mediante los operadores catchError y of de RxJS, garantizando una respuesta bilingüe al usuario incluso si se supera la cuota de peticiones de la API (Error 429) o hay fallos de red.

En resumen, Dada la naturaleza de las cuotas de las APIs gratuitas o de pago por uso, se ha diseñado un sistema de seguridad basado en operadores de **RxJS**:

- **Manejo del Error 429 (Too Many Requests):** Si se supera la cuota, el operador catchError intercepta la excepción antes de que rompa la aplicación.
- **Respuestas de Respaldo:** Mediante el operador of(), el sistema devuelve una respuesta predefinida y bilingüe (gestionada con TranslateService). Esta respuesta informa al usuario de la alta demanda y ofrece soluciones alternativas o preguntas frecuentes extraídas localmente, manteniendo la utilidad del asistente incluso sin conexión con Google.

a diferencia del uso de SDKs, externos que pueden inflar el *bundle* final de la aplicación, se ha optado por una **implementación nativa mediante HttpClient** de Angular.

- **Protocolo:** Comunicación vía REST a través de un endpoint.
- **Seguridad:** La API Key se gestiona mediante variables de entorno (de vercel), evitando su exposición en el repositorio de código fuente.
- **Encapsulamiento:** El servicio utiliza un flujo de datos basado en Observable<string>, permitiendo que cualquier componente de la aplicación pueda consumir respuestas de la IA de forma reactiva.

Para evitar las "alucinaciones" (respuestas inventadas) y asegurar que el bot actúe solo como experto en CuidaDos, se ha implementado una **Instrucción de Sistema (System Instruction)**.

- **Estrategia:** Se pre-inyecta un bloque de configuración en cada petición que define la personalidad (asistente de cuidados), el tono (empático y profesional) y la base de conocimiento (servicios de la plataforma, horarios y tipos de usuarios).
- **Restricción:** Se instruye al modelo para que decline peticiones no relacionadas con la salud, la asistencia domiciliaria o el funcionamiento de la web.

Uno de los mayores retos técnicos en la integración de IA es la sincronización de la respuesta con la interfaz de usuario.

- **NgZone y Change Detection:** Las respuestas asíncronas de APIs externas a veces pueden ejecutarse fuera de la "zona" de Angular. Se ha implementado `NgZone.run()` y `ChangeDetectorRef.detectChanges()` para forzar la actualización del DOM en el momento exacto en que llega el paquete de datos, garantizando que el usuario vea el texto aparecer de forma fluida y sin retardos visuales.
- **Estado de Carga:** Se utiliza un *signal* de estado para gestionar la experiencia de usuario (esqueletos de carga o indicadores de escritura).

4.7. Sistema de Alertas y Notificaciones: Telegram Bot API

Este componente es un sistema de mensajería unidireccional (notificaciones) diseñado para alertar al equipo de administración.

Se ha optado por Telegram frente al correo electrónico convencional debido a su baja latencia y la capacidad de ofrecer notificaciones instantáneas en dispositivos móviles sin coste operativo.

La comunicación no se realiza mediante librerías de terceros, sino a través de llamadas directas a la API REST de Telegram, optimizando así el rendimiento de la aplicación.

- **Endpoint:** Se consume el recurso `https://api.telegram.org/bot<token>/sendMessage`.
- **Seguridad y Configuración:** Tanto el Bot Token como el Chat ID (identificador del grupo de administradores) se encuentran centralizados en las variables de

entorno (environment.ts). Esto permite cambiar el destino de las alertas entre entornos de desarrollo y producción sin modificar el código fuente.

Función: Envío de mensajes estructurados desde la aplicación hacia un chat o canal privado de administración.

Justificación: Se prefiere esta solución sobre los sistemas de chat internos para asegurar que los administradores reciban avisos instantáneos en sus dispositivos móviles sin necesidad de mantener la aplicación CuidaDos abierta en el navegador.

Para garantizar que el sistema de mensajes sea privado y seguro, se han seguido los siguientes protocolos:

- **Identificación:** El bot solo envía mensajes al Chat ID configurado en las variables de entorno, lo que impide que la información se filtre a otros grupos o usuarios.
- **Tokenización:** El Bot Token está oculto en el servidor y en los archivos de configuración de entorno, protegiendo el acceso a la cuenta del bot.

5. GUÍA DE INSTALACIÓN (ENTORNO LOCAL)

5.1 Instalación de Prerrequisitos (Paso a Paso)

Antes de desplegar el proyecto, es necesario preparar el entorno en la máquina anfitriona.

Paso 1: Instalación de Node.js y NPM

Node.js es el entorno de ejecución. Al instalarlo, se incluye automáticamente NPM.

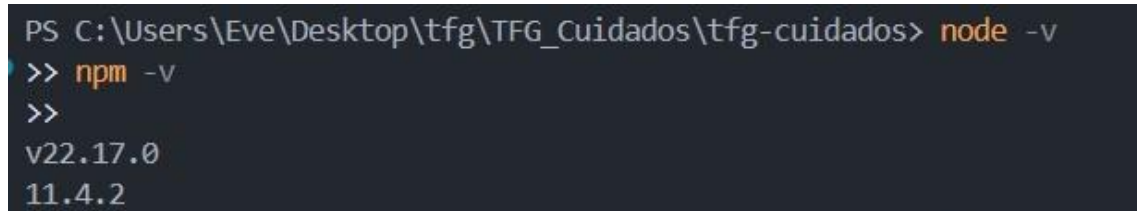
1. Acceder al sitio oficial: <https://nodejs.org/>
2. Descargar la versión **LTS (Long Term Support)** recomendada (v18.13.0 o superior).
3. Ejecutar el instalador siguiendo el asistente predeterminado.

Comando de verificación: Abra una terminal (PowerShell o CMD) y ejecute:

node -v npm

-v

(Si escoge las versiones actuales, debe devolver versiones superiores a v22.x y v11.x respectivamente).



```
PS C:\Users\Eve\Desktop\tfg\TFG_Cuidados\tfg-cuidados> node -v
v22.17.0
>> npm -v
11.4.2
```

Ilustración 17: Verificación instalación node.js

Paso 2: Instalación de Git

Necesario para clonar el repositorio y gestionar versiones.

1. Descargar el instalador desde: <https://git-scm.com/downloads>
2. Instalar manteniendo las opciones por defecto (especialmente la opción "Git from the command line").

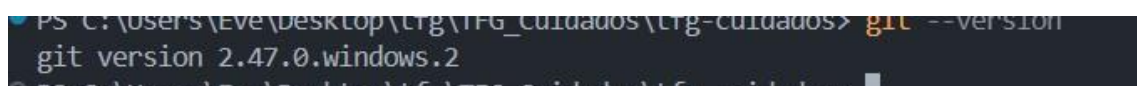
Configuración inicial (Comandos): Una vez instalado, ejecute en la terminal para configurar su identidad:

```
git config --global user.name "Tu Nombre" git
```

```
config --global user.email "tu@email.com"
```

Comando de verificación:

```
git --version
```



```
PS C:\Users\Eve\Desktop\tfg\TFG_Cuidados\tfg-cuidados> git --version
git version 2.47.0.windows.2
```

Ilustración 18: Instalación de git.

Paso 3: Instalación de Angular CLI (v21)

Esta herramienta se instala como un paquete global de NPM. Es el único paso que se realiza íntegramente por consola.

Comando de instalación: Ejecute el siguiente comando en la terminal con permisos de administrador (o usar sudo en Mac/Linux):

```
npm install -g @angular/cli@21
```

(Se especifica @21 para garantizar la compatibilidad con el proyecto, aunque @latest suele funcionar).

Comando de verificación: ng
version



Ilustración 19: Versión Angular.

Paso 4: Instalación de Dependencias del Proyecto

Una vez instaladas las herramientas globales, se debe instalar las librerías específicas de **cuidaDos** (Supabase, Material, Tailwind, etc.).

1. Navegue hasta la carpeta del proyecto descargado:

```
cd ruta/a/tfg-cuidados
```

2. Ejecute el comando de instalación automatizada:

```
npm install
```

(Este comando leerá el archivo package.json e instalará todas las dependencias exactas en la carpeta node_modules).

5.2 Entorno de desarrollo

Siga estos pasos para levantar el entorno de desarrollo:

Paso 1: Clonado del Repositorio

```
git clone https://github.com/[TU_USUARIO]/tfg-cuidados.git cd  
tfg-cuidados
```

Paso 2: Instalación de Dependencias El proyecto utiliza un package-lock.json para asegurar versiones exactas.

```
npm install
```

Paso 3: Configuración de Variables de Entorno Angular requiere las credenciales de conexión. Cree o modifique el archivo src/environments/environment.ts: TypeScript

```
export const environment = {  
  production: false,  
  // URL del proyecto Supabase (Project Settings > API)  
  supabaseUrl: 'https://xyzcompany.supabase.co',  
  // Clave Anónima Pública (Project Settings > API > anon public)  
  supabaseKey: 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...'  
};
```

Paso 4: Ejecución npm

```
start
```

El aplicativo estará disponible en <http://localhost:4200/>.

6. GUÍA DE DESPLIEGUE (ENTORNO CLOUD - VERCEL)

El despliegue en producción se gestiona a través de **Vercel**, aprovechando su red CDN global.

6.1 Configuración de Build

Vercel detecta automáticamente el framework Angular. Asegúrese de que la configuración de salida sea correcta:

- **Build Command:** npm run build
- **Output Directory:** dist/tfg-cuidados/browser (Importante: Angular 17+ y 21 cambian la carpeta de salida a /browser cuando se usa SSR).

6.2 Gestión de Rutas (SPA)

Para evitar errores 404 al recargar la página en rutas profundas (ej:

/dashboard/contratos), se incluye el archivo de configuración vercel.json en la raíz:

JSON

```
{  
  
  "rewrites": [  
  
    { "source": "/(.*)", "destination": "/index.html" }  
  
  ]  
  
}
```

6.3 Variables de Entorno en Producción

Por seguridad, **nunca** suba las claves al repositorio. Configúrelas en el panel de Vercel (*Settings > Environment Variables*):

- NG_APP_SUPABASE_URL: Valor de producción.
- NG_APP_SUPABASE_KEY: Valor de producción.

7. ARQUITECTURA DE BASE DE DATOS (SUPABASE)

El sistema utiliza **Supabase** (PostgreSQL) como motor de base de datos relacional. La arquitectura se ha diseñado para garantizar la integridad referencial y la seguridad de los datos mediante lógica del lado del servidor.

7.1 Aislamiento de Capas y Seguridad de Esquemas

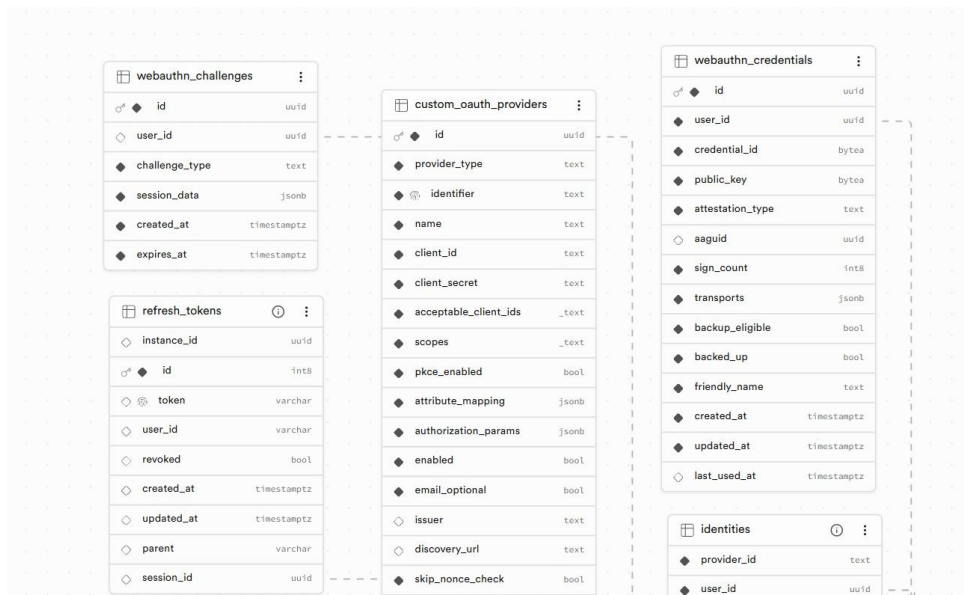
La base de datos se divide en dos áreas de responsabilidad:

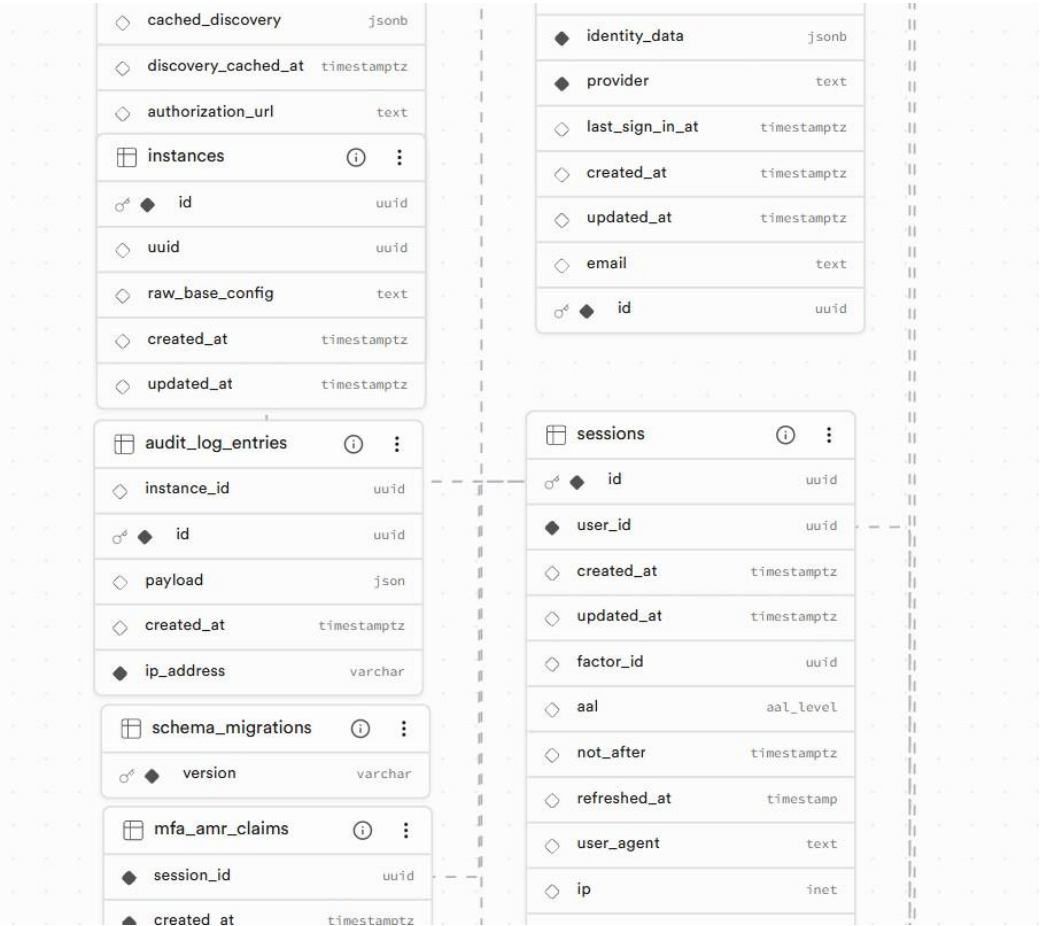
- **Capa de Autenticación (auth):** Gestionada por Supabase, es de solo lectura a través del panel principal para proteger las credenciales de los usuarios. Aquí residen los disparadores de pre-confirmación y creación inicial.
- **Capa de Aplicación (public):** Donde reside toda la lógica de negocio, servicios y contratos. Los disparadores en esta capa aseguran que los datos del frontend cumplan con las reglas de integridad antes de ser persistidos

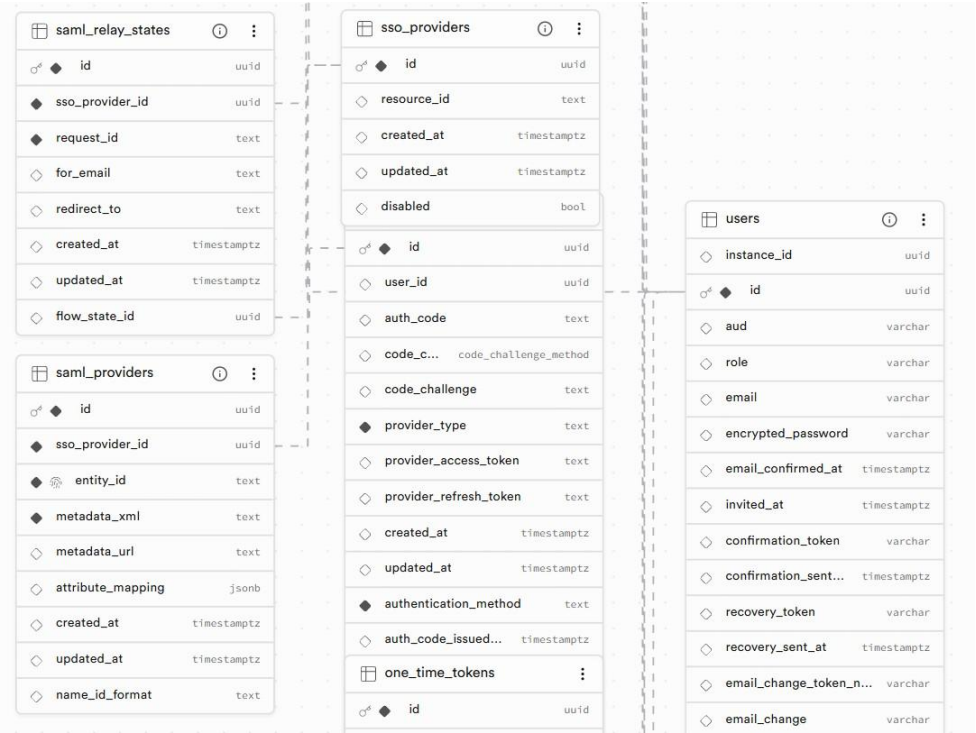
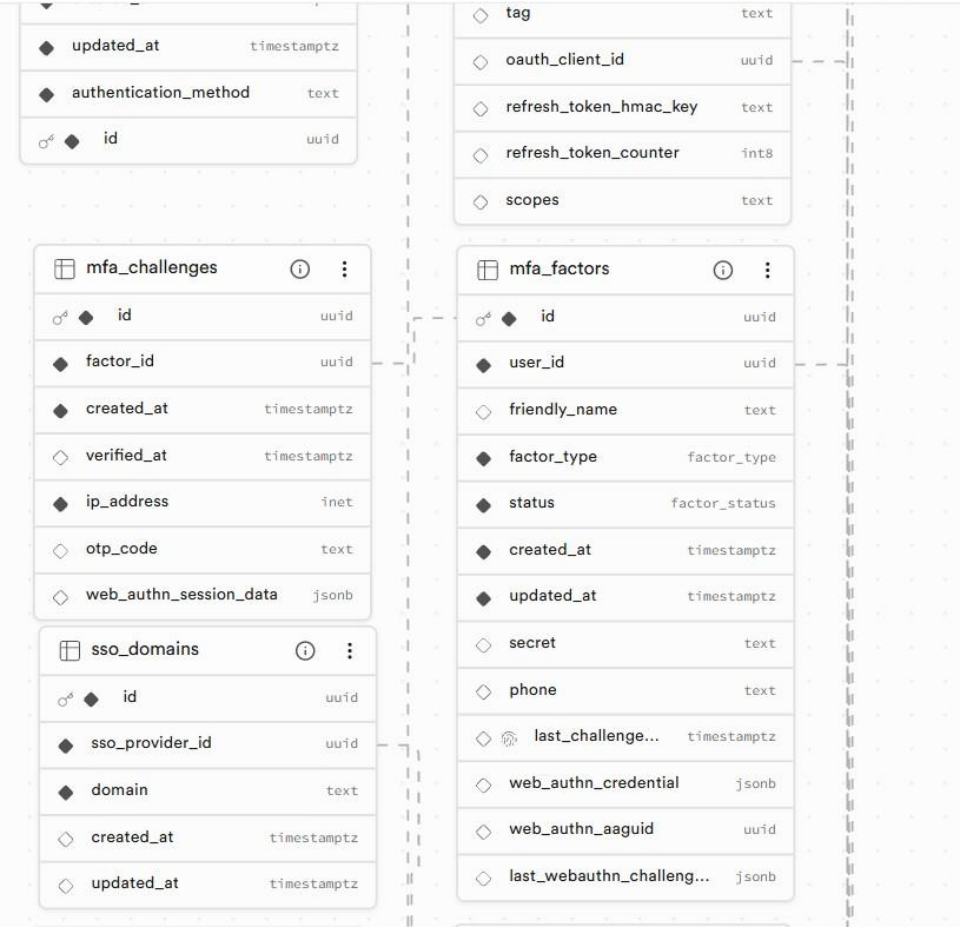
7.2 Modelo Relacional y Especialización de roles

La gestión de usuarios se divide en dos capas para separar la autenticación de la información de negocio:

- **Esquema auth (Interno):** Almacena las credenciales de acceso (email y contraseña) gestionadas por Supabase Auth.







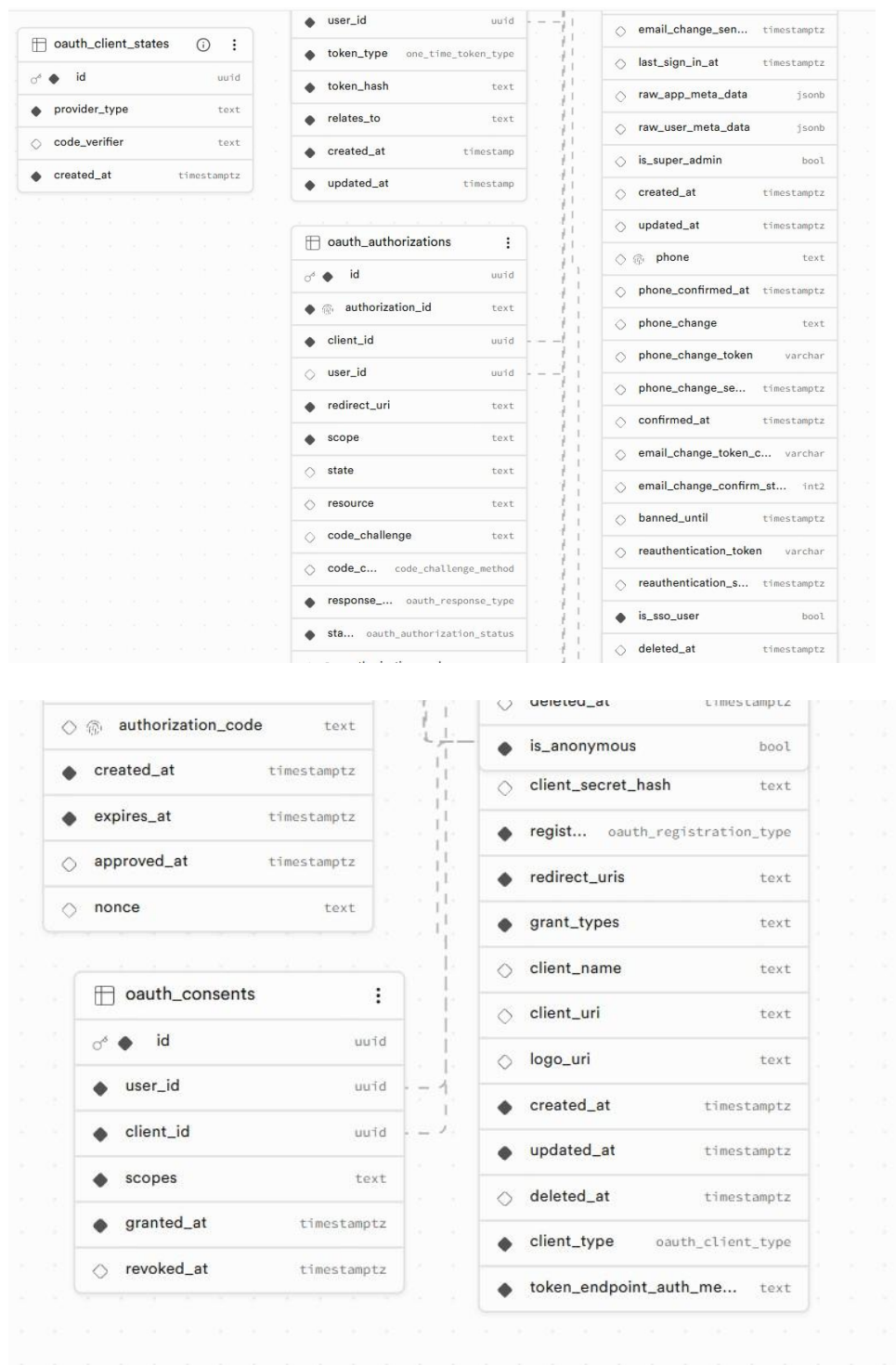


Ilustración 20: Esquema Auth Supabase.

- **Esquema public (Negocio):** La tabla public.User_public extiende el perfil del usuario.

- o **Atributo rol:** Campo crítico que define los permisos del sistema mediante una especialización: Cliente (Client) , Empresa (Business) o Administrador (Administrator).



Ilustración 21: Esquema Public Supabase.

7.2.1 Diagrama Entidad-Relación (ERD)

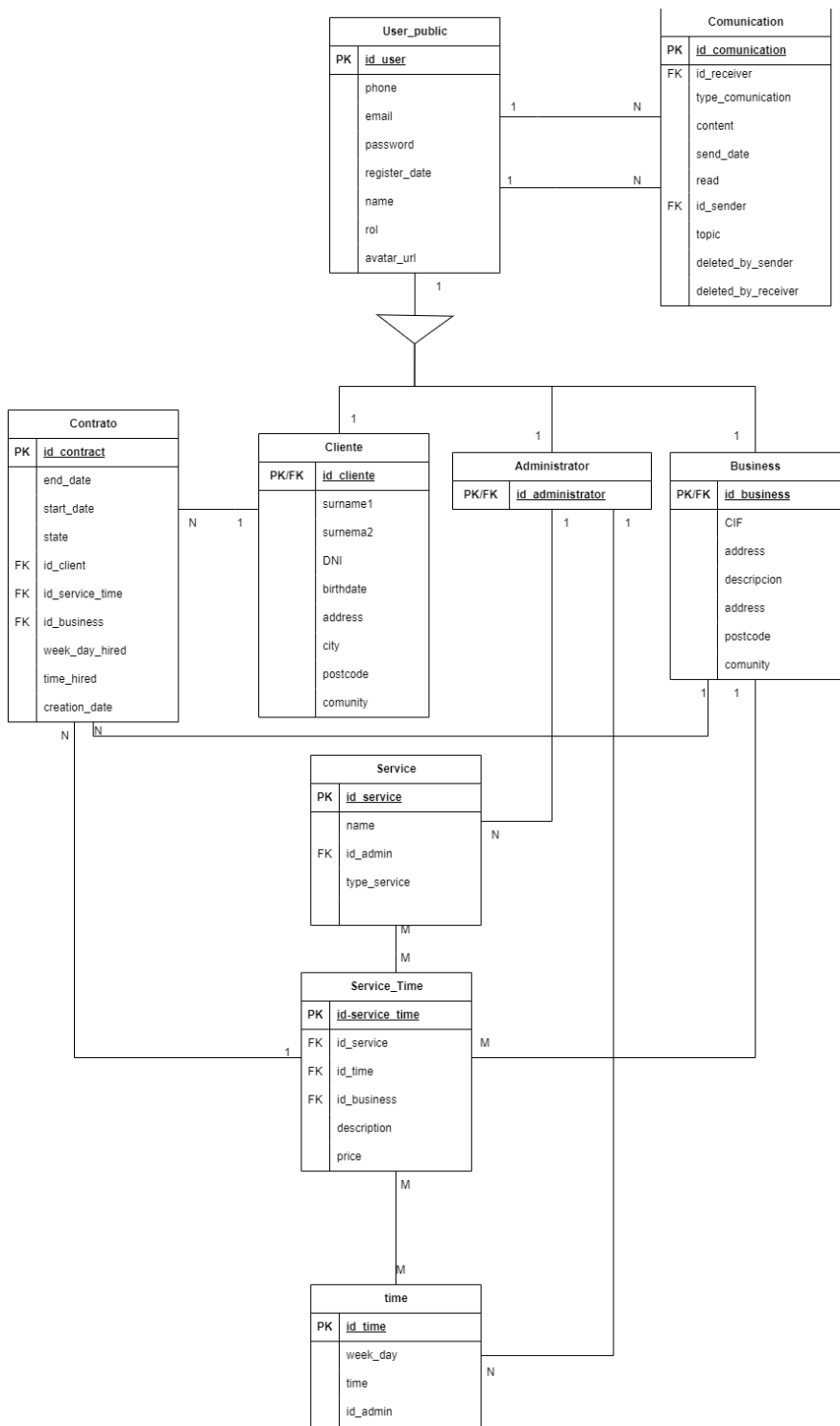


Ilustración 22: MER cuidaDos.

Justificación del diseño:

El diseño planteado persigue representar de manera adecuada el proceso de prestación de servicios bajo un horario determinado (día de la semana y hora) y su correspondiente contratación por parte de los clientes.

Los roles que interactúan con el sistema son:

- Cliente, que solicita o contrata servicios.
- Empresa, que ofrece dichos servicios.
- Administrador, que supervisa la información del sistema.

El diseño se ha enfocado en mantener la coherencia entre las entidades principales (empresa, cliente, servicio, horario y contrato), garantizando al mismo tiempo claridad en las relaciones y facilidad de consulta.

Supuestos:

- La base de datos propuesta es sencilla, pero refleja de manera adecuada la estructura lógica del aplicativo.
- Al ser un sistema nuevo, se espera un crecimiento progresivo, lo que implicará la creación futura de nuevas tablas (por ejemplo, *historiales*, *incidencias*, *pagos*, etc.).
- El diseño actual es escalable: permite incorporar más atributos a las entidades existentes o agregar nuevas entidades sin comprometer la integridad del modelo.

Principales decisiones:

La decisión principal fue la creación de la tabla Contrato, que actúa como nexo entre clientes, empresas y servicios ofrecidos en horarios específicos. De este modo:

- Las empresas pueden conocer qué contratos tienen activos y en qué horarios prestan sus servicios.
- Los clientes pueden visualizar los servicios contratados y cuándo serán prestados.

Esto resuelve la representación de la actividad simultánea tanto en el calendario del cliente como en el de la empresa.

Otra de las decisiones fue si la tabla intermedia entre servicio y horario iba a tener como clave primaria una clave compuesta por `id_service` e `id_time`. Al final se creó una clave primaria artificial autoincremental `id_service_time` y como FK `id_time` e `id_service`, como combinación única.

Ventajas de mi elección (PK artificial + UNIQUE):

1. Facilidad de referenciar
2. Mejor rendimiento en JOINS: los índices sobre un solo campo INTEGER son más eficientes y las búsquedas son más rápidas
3. Integridad mantenida
 - La restricción UNIQUE (`id_service`, `id_time`) sigue garantizando que no haya duplicados.

Finalmente, otra de las decisiones importantes fue el cambio de los atributos precio y descripción del servicio al `servicio_horario`, ya que se determinó que, un administrador podría crear un servicio y las empresa podría elegir entre un catálogo de servicios, pero era la empresa quien establecía la descripción y el precio para el servicio que esta ofrecía.

Dificultades y resolución:

1. Representación de las actividades

La necesidad de reflejar “cuándo y con quién” se realiza un servicio llevó a crear la tabla Contrato, que integra la información del cliente, la empresa y el horario contratado.

2. Cambios de horario y su impacto en los contratos

Inicialmente, el horario del servicio (día y hora) dependía de las tablas `Servicio_Horario` y `Horario`. Esto provocaba un problema: si el horario cambiaba, afectaba a todos los contratos relacionados, lo que no es deseable.

Para resolverlo, se añadieron los campos:

- `day_week_hired`
- `time_hired`

Estos campos representan una desnormalización controlada que viola la Tercera Forma Normal (3FN) debido a una dependencia transitiva (ya que la hora y el día dependen de `id_service_time`).

Sin embargo, esta desnormalización es intencionada y justificada por los siguientes motivos:

Motivo y descripción

Simplicidad de consultas → Evita realizar múltiples JOIN con Service_Time y Time.

Estabilidad de datos históricos → Los contratos conservan la hora y día tal como se firmaron, aunque cambien los horarios de los servicios.

Rendimiento → Reduce el número de consultas anidadas, mejorando la velocidad de lectura.

Lógica de negocio → Los contratos son registros cerrados e inmutables, por lo que la redundancia no genera inconsistencias.

En una futura iteración del proyecto, se contempla la creación de una tabla Contract_copy (Anexo I) que almacene los valores originales de hora y día, cumpliendo con la 3FN. Sin embargo, por simplicidad y alcance actual del sistema, la prioridad de dicha migración es baja.

Finalmente, otra de las decisiones importantes fue el cambio de los atributos precio y descripción del servicio al servicio_horario, ya que se determinó que, un administrador podría crear un servicio y las empresa podría elegir entre un catálogo de servicios, pero era la empresa quien establecía la descripción y el precio para el servicio que esta ofrecía.

Dificultades y resolución:

1. Representación de las actividades

La necesidad de reflejar “cuándo y con quién” se realiza un servicio llevó a crear la tabla Contrato, que integra la información del cliente, la empresa y el horario contratado.

3. Crecimiento de la tabla Service_Time

Cada combinación de servicio y horario se representa como una tupla independiente en la tabla Service_Time. Aunque esto puede generar un gran número de registros a futuro, la estructura es eficiente y clara:

- Facilita las consultas directas.
- No afecta el rendimiento ni la integridad del modelo.
- Mantiene la legibilidad y escalabilidad del sistema.

Política de modificación de contratos

Los contratos se consideran registros cerrados e inmutables. No se permite su modificación una vez creados; si se desea cambiar alguna condición (horario, servicio, precio, etc.), se crea un nuevo contrato y se marca el anterior como *finalizado* o *cancelado*. Esta política asegura:

- Integridad de la información contractual.
- Historial completo de acuerdos firmados.
- Simplicidad en la gestión de versiones y trazabilidad de los cambios.

4. Campo rol en la tabla User_Public.

La decisión de incluir un campo explícito rol (o discriminador) en la tabla principal de Usuario, en lugar de depender exclusivamente de la existencia del usuario en tablas separadas (Cliente, Empresa, Administrador), representa una optimización técnica profunda.

A continuación, se detalla y justifica exhaustivamente esta elección arquitectónica desde una perspectiva de rendimiento, seguridad y mantenibilidad, redactada en tercera persona para su posible integración en documentación técnica o académica.

1. Optimización Crítica de las Políticas de Seguridad (RLS en Supabase)

El sistema basa su infraestructura de backend en Supabase y PostgreSQL, haciendo uso intensivo de la Seguridad a Nivel de Fila (RLS, por sus siglas en inglés). Las políticas RLS se ejecutan en la base de datos por cada fila que un usuario intenta leer, insertar, modificar o eliminar.

Si el rol del usuario no está definido de forma explícita en la tabla Usuario, el motor de la base de datos se vería obligado a ejecutar subconsultas complejas (por ejemplo, comprobar con un EXISTS si el identificador del usuario se encuentra en la tabla Empresa, y luego en la tabla Cliente, etc.) para cada operación. Al centralizar el atributo rol en la tabla Usuario, la política RLS se simplifica a una evaluación condicional directa y atómica (ej. rol = 'Empresa'). Esto reduce drásticamente el coste computacional, evita cuellos de botella en el rendimiento y garantiza respuestas en milisegundos, algo vital cuando el volumen de datos (servicios, contratos, comunicaciones) comience a escalar.

2. Eficiencia en la Capa de Presentación (Angular y Gestión de Estado)

La arquitectura del cliente (Frontend) está desarrollada como una Single Page Application (SPA) en Angular, gestionando la navegación mediante Guards (rutas protegidas) e inyección de dependencias.

Cuando un individuo inicia sesión, el sistema necesita determinar de manera inmediata qué interfaz debe cargar (el Panel de Administración, el Panel de Empresa o el Perfil del Cliente). Si la base de datos careciera de un campo rol en la entidad principal, el AuthService de Angular tendría que realizar múltiples peticiones asíncronas para "adivinar" el tipo de usuario, o el backend tendría que ensamblar una respuesta artificial consultando varias tablas. Al disponer del campo rol directamente en el perfil base del usuario, el sistema obtiene esta información crucial en la primera y única petición de autenticación. Esto permite que los Signals de Angular actualicen el estado de la sesión instantáneamente y que el enrutador dirija al usuario a su panel correspondiente sin latencia perceptible.

3. Implementación del Patrón de Diseño de Base de Datos (Discriminador)

Desde el punto de vista del modelado de datos, las entidades Cliente, Empresa y Administrador comparten atributos y comportamientos comunes (credenciales, correo electrónico, teléfono, fecha de registro). Esto conforma un clásico esquema de herencia.

En bases de datos relacionales, al mapear una herencia mediante el patrón de "Tabla por Tipo" (donde existe una tabla padre y varias tablas hijas), es una práctica recomendada incluir una columna "discriminadora" en la tabla padre. El campo rol actúa exactamente como este discriminador. Su presencia indica inequívocamente a qué tabla hija (Cliente, Empresa o Administrador) pertenece el resto de la información específica de esa entidad. Esto elimina la ambigüedad y permite a los desarrolladores escribir consultas de combinación (JOIN) dinámicas y mucho más limpias, sin depender de lógicas deductivas frágiles.

4. Escalabilidad y Flexibilidad Futura

El diseño de software debe anticipar la evolución del producto. Centralizar la asignación del rol en un atributo de la entidad Usuario dota al sistema de una flexibilidad superior.

Si en futuras iteraciones del proyecto CuidaDos se requiere añadir nuevos tipos de usuarios (por ejemplo, "Auditor", "Soporte Técnico" o "Gestor de Contenidos"), bastará con añadir un nuevo valor al campo rol (idealmente tipado como un ENUM en PostgreSQL). El sistema no requerirá la creación forzosa de nuevas tablas a menos que esos nuevos roles necesiten almacenar información

estructuralmente distinta. Además, si en algún escenario un usuario necesitara cambiar de rol o tener un rol transitorio, la operación consistiría en una simple actualización de un campo de texto, en lugar de un complejo proceso de migración de registros entre distintas tablas.

7.3 Lógica de Negocio

7.3.1 Lógica de Servidor: Triggers y Funciones RPC

El sistema delega la validación y la integridad de datos al motor de base de datos mediante funciones y disparadores. Esto garantiza que las reglas de negocio se cumplan incluso si se intenta manipular la API externamente.

A. Gestión de Usuarios y Perfiles

- **handle_new_user:** Trigger que automatiza la creación del perfil en la tabla `public.usuarios` inmediatamente después de un registro exitoso en `auth.users`.
- **update_profile_complete:** Procedimiento robusto para actualizar todos los campos del perfil (dirección, localidad, descripción, etc.) en una sola transacción, asegurando la consistencia de los datos del usuario.
- **activar_usuario_al_confirmar:** Gestiona el estado de activación del usuario una vez que se verifica su identidad.

B. Seguridad y Control de Acceso (RBAC)

- **is_admin:** Función de seguridad crítica que verifica si el UUID del usuario solicitante posee el rol de "Administrador" en la tabla de perfiles. Se utiliza como filtro en las políticas RLS y funciones sensibles.
- **check_user_exists** y **email_exists:** Funciones auxiliares para validar la disponibilidad de credenciales durante el registro o actualización, evitando duplicidad de datos.
- **eliminar_usuario_total:** Función de limpieza de los usuarios, elimina un usuario de forma lógica, estableciendo el estado del usuario a `FALSE`, de forma que no podrá conectarse al aplicativo al iniciar sesión, no se le tendrá en cuenta en el Dashboard del usuario y no podrá recuperar su cuenta (solo crearse otra

cuenta nueva con otro email). Esta función fue validada con un 100% de éxito en las pruebas de Postman.

C. Automatización de Pruebas E2E

Para facilitar el testing continuo con **Cypress** y **Postman**, se han implementado mecanismos que evitan el bloqueo por flujos manuales:

- **pre_confirm_test_users**: Trigger que intercepta registros con dominios de prueba (ej. @test.com) para auto-confirmar el correo electrónico.
- **handle_test_user_confirmation**: Asegura que el flujo de confirmación para cuentas de prueba se complete sin intervención humana.

D. Integración de Limpieza y Contratos

- **rellenar_contrato_desde_angular**: Gestiona la lógica de inserción de contratos, validando parámetros antes de persistirlos en la base de datos.
- **limpieza_al_desactivar_usuario** y **limpieza_total_antes_borrado**: Garantizan que, al eliminar o desactivar una cuenta, se limpien en cascada las relaciones (mensajes, servicios, contratos), cumpliendo con las normativas de protección de datos y manteniendo la base de datos optimizada.

E. Triggers del Esquema auth (Gestionados por Supabase) Estos

disparadores reaccionan a eventos de autenticación y registro.

Trigger	Tabla	Función Asociada	Evento	Propósito
on_auth_user_created_pre_confirmation	users	pre_confirm_test_users	BEFORE INSERT	Auto-confirmar correos @test.com para testeo.
on_auth_user_created	users	handle_new_user	AFTER INSERT	Crear el perfil público tras el registro.

on_auth_user_created_test	users	handle_test_user_confirmation	AFTER INSERT	Flujo de confirmación interna para usuarios de prueba.
on_auth_user_verified	users	activar_usuario_al_confirmar	AFTER UPDATE	Activar cuenta cuando el email es verificado.

Tabla 5: Triggers Auth.

F. Triggers del Esquema public (Lógica de Negocio)

Estos disparadores aseguran la integridad de los datos de la aplicación.

Trigger	Tabla	Función Asociada	Evento	Propósito
tr_rellenar_contrato_angular	Contrato	rellenar_contrato_desde_angular	BEFORE INSERT	Validar y completar datos del contrato antes de guardar.
tr_reaccion_desactivar_usuario	Usuario	limpieza_al_desactivar_usuario	AFTER UPDATE	Limpiar datos vinculados al

Tabla 6: Triggers Public.

7.3.2 Seguridad (Row Level Security - RLS)

El sistema implementa políticas de seguridad a nivel de fila para garantizar que el acceso a la información esté estrictamente regulado según el rol y la relación del usuario con los datos.

A. Visibilidad de Perfiles y Catálogo

1. Usuarios (Clientes y Empresas): Los perfiles son visibles para todos los usuarios, pero únicamente si están autenticados en la plataforma. Esto permite que

tanto clientes como empresas consulten la información necesaria para formalizar contratos y servicios.

2. Servicios y Horarios: Estas tablas actúan como un catálogo gestionado por los Administradores. Las Empresas tienen permiso para seleccionar y vincular estos servicios y horarios predefinidos a su oferta particular.

3. Servicios por Empresa: La gestión de la tabla servicios_horarios es responsabilidad de las empresas, pero el RLS restringe el acceso de modo que cada empresa solo puede gestionar sus propios registros.

B. Privacidad en Comunicaciones y Contratos

1. Mensajería Interna: La comunicación solo es accesible para las partes implicadas (emisor y receptor). Si una de las partes decide borrar el mensaje, este dejará de ser visible para ella, pero seguirá siendo accesible para la otra parte implicada que no lo haya eliminado.

2. Contratos: El acceso está limitado exclusivamente a las partes involucradas en el acuerdo. No obstante, un contrato deja de ser visible si se marca como inactivo para una de las partes, reforzando el control individual sobre la información histórica

7.4 Seguridad Avanzada y Ejecución de Funciones (Security Definer)

Para garantizar que ciertas operaciones críticas se realicen correctamente sin comprometer la seguridad general del sistema, se ha utilizado el modificador SECURITY DEFINER en funciones específicas como eliminar_usuario_total, update_profile_complete e is_admin.

¿Por qué es necesario?

Por defecto, las funciones en PostgreSQL se ejecutan como SECURITY INVOKER, lo que significa que tienen los mismos permisos que el usuario que las llama. Sin embargo, en CuidaDos se necesita:

1. Bypass de RLS: Un usuario normal no tiene permisos para borrar filas en esquemas protegidos o realizar limpiezas totales. Al definir la función como

SECURITY DEFINER, la función se ejecuta con los privilegios del creador de la función (admin) y no del usuario actual.

2. Gestión de Autenticación: La función `eliminar_usuario_total` necesita interactuar con el esquema `auth`, el cual está restringido. Solo una función con privilegios elevados puede coordinar el borrado simultáneo en `auth.users` y `public.usuarios`.
3. Validación de Roles: La función `is_admin` requiere consultar tablas de sistema para verificar permisos; el uso de este modificador asegura que la validación sea infalible independientemente de quién la invoque.

8. EJECUCIÓN Y PRUEBAS DEL APLICATIVO

8.1 Comandos de Ciclo de Vida

El archivo `package.json` define los siguientes scripts:

- **npm start:** Levanta el servidor de desarrollo (HMR activo).
- **npm run build:** Compila la aplicación para producción con optimización AOT (Ahead-of-Time).
- **npm test:** Ejecuta la suite de pruebas unitarias con Karma.

8.2 Gestión de Bundles y Optimización (Caso de Estudio)

Durante el proceso de compilación para producción (`ng build`), se realiza un análisis del peso de los archivos JavaScript generados. Angular establece por defecto unos "presupuestos" (*Budgets*) muy restrictivos (1MB máximo) para garantizar el rendimiento en redes móviles 3G.

1. El Desafío (Problema Identificado)

Al integrar librerías robustas necesarias para la lógica de negocio, el tamaño del *bundle* inicial excedió el límite predeterminado, generando el siguiente error de compilación:

Error: bundle initial exceeded maximum budget. Budget 1.00 MB was not met by 313.55 kB.

Este incremento de tamaño se debe a la inclusión de dependencias críticas en el hilo principal:

- **@supabase/supabase-js:** Cliente completo para la conexión a base de datos y tiempo real.
- **@angular/material:** Componentes de interfaz de usuario complejos (tablas, diálogos).
- **Chart.js:** Motor de renderizado de gráficas para el panel de administración.

2. Solución Implementada

Para solucionar el bloqueo sin sacrificar funcionalidad, se reconfiguraron los umbrales de rendimiento en el archivo `angular.json`. Se estableció una política de presupuestos realista para una **Aplicación de Gestión (Enterprise SPA)**, diferenciándola de una web estática ligera.

Configuración aplicada (`angular.json`):

```
"budgets": [  
  {  
    "type": "initial",  
    "maximumWarning": "2MB",  
    "maximumError": "4MB"  
  }  
]
```

Tabla 7: Ajustes Test.

3. Justificación Técnica y Estrategia de Carga

A pesar de aumentar el tamaño permitido del archivo principal (`main.js`), el rendimiento de la aplicación se mantiene óptimo gracias a la implementación de dos estrategias compensatorias:

- 1. **Lazy Loading (Carga Perezosa):** Como se observa en la salida de la compilación (Figura X), no todo el código se carga al inicio. Módulos como la "Recuperación de Contraseña" se han separado en *chunks* diferidos (chunkrecover-password.js), que solo se descargan si el usuario los solicita.
- 2. **SSR (Server-Side Rendering):** Gracias al renderizado en servidor, el usuario visualiza la interfaz (FCP) antes de que termine de descargarse el bundle de JavaScript completo, mitigando la sensación de espera.

4. Resultado

```
PS C:\Users\Eve\Desktop\tfg\TFG_Cuidados\tfg-cuidados> npm run build

> tfg-cuidados@0.0.0 build
> ng build

Browser bundles
Initial chunk files | Names | Raw size | Estimated transfer size
main-BERFKOR2.js | main | 701.27 kB | 156.18 kB
chunk-ZHKNJ2QP.js | - | 556.88 kB | 138.08 kB
styles-N56PI5UO.css | styles | 55.40 kB | 6.95 kB
Initial total | 1.31 MB | 301.21 kB

Lazy chunk files | Names | Raw size | Estimated transfer size
chunk-XHJCSMPV.js | recover-password | 3.04 kB | 1.30 kB

Server bundles
Initial chunk files | Names | Raw size
server.mjs | server | 1.26 MB
main.server.mjs | main.server | 1.15 MB
chunk-3AZZQSYJ.mjs | - | 576.62 kB
polyfills.server.mjs | polyfills.server | 233.25 kB
chunk-L2MVFUKD.mjs | - | 1.32 kB
chunk-T55IDOPT.mjs | - | 976 bytes

Lazy chunk files | Names | Raw size
chunk-RHDGF2TI.mjs | xhr2 | 12.14 kB
chunk-N3SI5I4Q.mjs | xhr2 | 12.07 kB
chunk-74CEVUQZ.mjs | recover-password | 3.10 kB

Prerendered 19 static routes.
Application bundle generation complete. [11.481 seconds] - 2026-02-01T09:26:12.835Z
```

Tabla 8: Resultado test tras configuración.

Como se observa en la salida, el sistema genera dos tipos de bundles:

- 1. **Browser Bundles:** Archivos JavaScript optimizados para el cliente.

2. **Server Bundles:** Archivos generados para el SSR (Server-Side Rendering).

9. ARQUITECTURA DEL PROYECTO (ESTRUCTURA DE CARPETAS)

El proyecto sigue una arquitectura modular basada en las recomendaciones de estilo oficiales (*Angular Style Guide*), agrupando los archivos por funcionalidad (Featurebased).

Al utilizar la arquitectura de Standalone Components (nativa en Angular 21), se ha eliminado la complejidad de los NgModules, resultando en un árbol de directorios limpio y escalable situado en `src/app`:

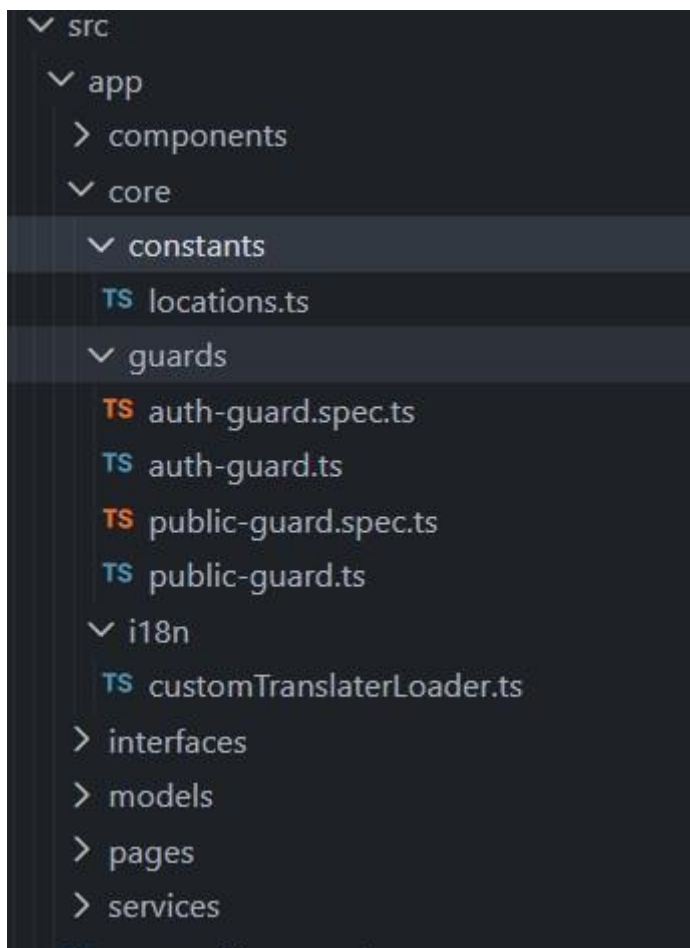


Ilustración 23: Arquitectura de carpetas.

- **Carpeta components:** Contiene los **Componentes de Presentación (Dumb Components)**.

Son elementos reutilizables de la interfaz (botones, tarjetas, tablas) que no dependen de la lógica de negocio ni de servicios externos. Reciben datos únicamente a través de `@Input` y comunican eventos al padre mediante `@Output`.

- **Carpeta constants.**

Almacena archivos con valores inmutables y de configuración global para evitar el uso de "números mágicos" o cadenas de texto dispersas.

- **Carpeta guards.**

Aloja la lógica de **Seguridad Perimetral del Enrutador**. Implementa las interfaces que interceptan la navegación antes de cargar una ruta. Aquí residen `AuthGuard` (verificación de sesión) y `PublicGuard` (rutas de acceso a los usuarios sin sesión).

- **Carpeta interfaces.**

Define los **Contratos de Datos (Data Contracts)** en TypeScript. Se utilizan para garantizar el tipado estricto de las respuestas de la base de datos (Supabase) y la estructura de los objetos que fluyen por la aplicación, facilitando el desarrollo y reduciendo errores en tiempo de ejecución.

- **Carpeta models.**

Contiene clases o tipos complejos que requieren lógica interna o transformación de datos. A diferencia de las interfaces (que son solo contratos), los modelos pueden incluir constructores o métodos auxiliares para formatear la información antes de mostrarla en la vista.

- **Carpeta services.**

Representa la **Capa de Lógica de Negocio y Acceso a Datos**.

Función: Son clases inyectables (Singleton) encargadas de la comunicación HTTP, la interacción con el cliente de Supabase y la gestión del estado reactivo (usando Signals). Los componentes delegan en ellos cualquier operación compleja.

10. NAVEGACIÓN Y SEGURIDAD EN RUTAS

La gestión de vistas en **cuidaDos** se delega al **Angular Router**, configurado mediante un array de objetos `Routes` que mapean URLs específicas a componentes *Standalone*.

Estrategia de Carga Híbrida

El sistema implementa una arquitectura de carga mixta para equilibrar el tiempo de inicio y la fluidez de navegación:

1. Carga Temprana (Eager Loading):

Los módulos críticos de uso frecuente (como Home, Dashboard, Contracts) se importan estáticamente. Al cargar estos recursos al inicio, las transiciones entre las opciones del menú principal son instantáneas (0ms de latencia), mejorando la experiencia de usuario (UX) en sesiones activas.

2. Carga Perezosa (Lazy Loading):

Se aplica en módulos aislados o de uso esporádico, como se evidencia en la ruta `recover-password`.

Implementación: Uso de la función `loadComponent` con importación dinámica (`import(...)`).

El código de recuperación de contraseña no se descarga en el navegador del usuario a menos que este lo solicite explícitamente, reduciendo el peso del *bundle* inicial.

Seguridad Perimetral (Guards)

La protección de rutas se aplica mediante Guardianes Funcionales (Functional Guards) inyectados en la propiedad `canActivate`:

- **authGuard:**

Protege las rutas privadas (`/home`, `/dashboard`, `/messages`, etc.). Verifica que exista una sesión válida en Supabase. Si el token ha expirado o no existe, bloquea la navegación y redirige al usuario al Login.

- **publicGuard:**

Protege la ruta raíz (`/`) y el Landing. Su función es inversa: si un usuario ya tiene sesión iniciada e intenta acceder al Landing, el guardián lo redirige automáticamente a su área personal (`/home`), evitando que tenga que volver a loguearse.

- **Resolución Estricta (runGuardsAndResolvers):**

En la ruta /register, se ha configurado la opción 'always'. Esto fuerza la re-ejecución de los validadores incluso si la navegación proviene de la misma ruta, asegurando que el formulario de registro se reinicie correctamente ante reintentos.

Mapa de Rutas (Route Map)

A continuación, se detalla la estructura de navegación definida en app.routes.ts:

“ ” (Raíz)	Público	Landing Page (Página de Aterrizaje).	publicGuard
about-us	Público	Página sobre información de los creadores del aplicativo y dirigente de la empresa cuidaDos.	-
contact	Público	Página para contacto con los administradores del aplicativo.	-
register	Público	Registro de Usuarios/Empresas.	-
manuals	Público	Manuales para los Usuarios y Empresas.	-
terms	Público	Términos y condiciones.	-
home	Privado	Vista Principal del Usuario.	authGuard
dashboard	Privado	Panel de Control (Admin).	authGuard
modifyprofile	Privado	Formulario para modificar los datos.	authGuard
managment-admin	Privado	Gestión de Usuarios (Admin).	authGuard
messages	Privado	Sistema de Mensajería Interna.	authGuard
contract	Privado	Gestión de Contratos.	authGuard

activities	Privado	Revisión de actividades (contratos: con quién se realiza/recibe la actividad, fecha y lugar) y gestión de la misma.	authGuard
notifications	Privado	Sistema de Notificaciones Interno.	authGuard
search-business	Privado	Búsqueda de servicios por categorías que ofrecen empresas y contrato.	authGuard
Services-directory	Privado	Categorías en las que se engloban los servicios.	authGuard
globalservices	Privado	Gestión de servicios por el administrador	authGuard
global-times	Privado	Gestión de horarios por el administrador.	authGuard
global-contracts	Privado	Gestión de contratos por el administrador.	authGuard
admin-services	Privado	Administración de servicios por parte de la empresa (ofertas).	authGuard
recover-password	Público	Recuperación (Lazy Loaded).	-
** (Wildcard)	-	Redirección por defecto a Raíz (Manejo 404).	-

Tabla 9: rutas.

11. Estrategia de Seguridad y Despliegue en Vercel (CI/CD)

Para el pase a producción de la plataforma "CuidaDos", se ha elegido el servicio de *hosting* Vercel, sincronizado directamente con el repositorio de GitHub.

Por motivos estrictos de seguridad y siguiendo las mejores prácticas de desarrollo, el archivo `environment.ts` (que contiene las claves de acceso a la base de datos de Supabase y a la API de Inteligencia Artificial de Google Gemini) fue añadido al `.gitignore` para evitar su exposición pública en el control de versiones.

Dado que Angular requiere la existencia física de este archivo durante el proceso de compilación (*build*), se ha diseñado e implementado un flujo de trabajo personalizado (CI/CD). Se ha desarrollado un *script* de pre-construcción en Node.js (`generate-env.js`) que se ejecuta automáticamente en los servidores de Vercel justo antes de compilar la aplicación.

Este *script* se encarga de:

1. Leer las variables de entorno inyectadas de forma segura desde el panel de administración de Vercel (`process.env`).
2. Generar dinámicamente y en tiempo real los archivos `environment.ts` y `environment.development.ts` necesarios.
3. Permitir que el compilador de Angular construya el empaquetado final sin errores.

Gracias a esta arquitectura, se garantiza que el código fuente público sea 100% seguro y libre de credenciales, manteniendo a su vez un proceso de despliegue continuo y automatizado cada vez que se realiza un nuevo *commit* en la rama principal.

12. DOCUMENTACIÓN

La documentación del proyecto CuidaDos se ha elaborado siguiendo los estándares de la ingeniería de software, garantizando que tanto los usuarios finales como futuros desarrolladores puedan comprender, operar y escalar la plataforma de manera eficiente. La estrategia de documentación se divide en varios niveles:

12.1. Documentación Externa (Manuales)

Se han redactado documentos específicos adaptados al perfil del lector:

- **Manual de Usuario:** Orientado a los usuarios finales (Clientes y Empresas). Detalla de forma visual y guiada los flujos de la aplicación (registro, búsqueda de servicios, contratación, mensajería y cancelación). Su enfoque es puramente funcional y de usabilidad.
- **Manual Técnico (Este documento):** Orientado a administradores de sistemas, arquitectos y desarrolladores. Recoge las decisiones de diseño arquitectónico, el stack tecnológico, el modelo relacional de la base de datos (Supabase) y los protocolos de despliegue en producción (Vercel).

12.2. Documentación Interna del Código (Clean Code y JSDoc)

El código fuente de la aplicación en Angular ha sido diseñado bajo los principios de *Clean Code* (Código Limpio), logrando que el sistema sea auto-descriptivo:

- **Tipado Estricto:** Gracias al uso avanzado de TypeScript y la creación de Interfaces y Models centralizados, las estructuras de datos están fuertemente tipadas, reduciendo la ambigüedad en el desarrollo.
- **Comentarios y JSDoc:** Las funciones complejas, especialmente en los servicios del núcleo (core/services), cuentan con bloques de documentación que describen los parámetros de entrada, el comportamiento esperado y el tipo de retorno.
- **Nomenclatura Semántica:** Las variables, métodos y componentes utilizan nombres descriptivos en inglés o español estandarizado, facilitando la trazabilidad de la lógica de negocio.

12.3. Documentación Gráfica y Diagramas

Para representar la arquitectura del sistema y el flujo de la información de manera visual, se han utilizado herramientas de modelado estandarizadas (como Draw.io). Se han generado diagramas de arquitectura que ilustran la interacción entre la capa de presentación (SPA en Angular), la capa de servicios (Inyección de dependencias) y el Backend as a Service (PostgreSQL y Supabase Auth).

12.4. Repositorio, README y Asistencia por IA

El control de versiones alojado en GitHub actúa como el punto de entrada principal para cualquier desarrollador.

- README.md: Contiene las instrucciones precisas para clonar, instalar las dependencias (npm install) y levantar el servidor de desarrollo (ng serve).
- Integración de Inteligencia Artificial: Como valor añadido y elemento de innovación tecnológica, la documentación incluye un enlace a un Asistente Virtual IA basado en modelos de lenguaje natural (LLM). Este agente ha sido entrenado específicamente con los manuales de CuidaDos, permitiendo a cualquier usuario o desarrollador hacer consultas técnicas interactivas sobre el proyecto sin necesidad de buscar manualmente en los PDFs.

13. MEJORAS FUTURAS

El desarrollo del proyecto **CuidaDos** se ha planteado bajo una metodología iterativa. Si bien la versión actual cumple con los objetivos principales establecidos para el Trabajo de Fin de Grado, la arquitectura *Serverless* diseñada (Angular + Supabase) está preparada para escalar. A continuación, se detallan las principales líneas de trabajo futuro para convertir la plataforma en un producto comercial completo:

- Integración de Pasarela de Pago y Modelo de Negocio
Actualmente, el acuerdo económico se realiza fuera de la plataforma. Como evolución natural, se propone la integración de una pasarela de pagos segura (como Stripe o PayPal). Esto permitiría retener los fondos de forma segura (sistema *escrow*) hasta que el servicio se haya prestado, ofreciendo garantías tanto al cliente como a la empresa, e introduciendo la posibilidad de monetizar la plataforma mediante el cobro de comisiones por transacción.
- Geolocalización Avanzada y Mapas Interactivos
Para mejorar la experiencia de búsqueda de los clientes, se propone implementar capacidades de geolocalización utilizando la extensión PostGIS nativa de la base de datos PostgreSQL en Supabase. Al combinar esto con la API de Google Maps o Mapbox en el frontend de Angular, los usuarios podrían buscar servicios asistenciales en un radio de kilómetros específico y visualizarlos en un mapa interactivo.
- Sistema de Valoraciones y Reputación
En el sector sociosanitario, la confianza es el pilar fundamental. Se plantea añadir un módulo de *reviews* donde los clientes puedan calificar (con estrellas de 1 a 5) y dejar comentarios públicos sobre las empresas o profesionales una vez finalizado un contrato. Esto generaría un ranking automatizado que premiaría a los mejores cuidadores en los resultados de búsqueda.

- Entrevistas por Videollamada (WebRTC)

Antes de contratar a alguien para el cuidado de un familiar, suele ser necesaria una entrevista visual. Implementar un módulo de videollamadas integrado directamente en el chat de la aplicación (utilizando tecnologías Peer-to-Peer como WebRTC) evitaría que los usuarios tengan que recurrir a plataformas de terceros (como Zoom o WhatsApp) para la toma de contacto inicial.

- Evolución a Aplicación Móvil o PWA (Progressive Web App)

Dado que los cuidadores y los familiares suelen necesitar consultar la plataforma en movimiento, el siguiente paso tecnológico sería evolucionar el frontend actual en Angular para transformarlo en una PWA, o utilizar *Ionic/Capacitor* para compilar la aplicación y publicarla en las tiendas nativas (App Store y Google Play). Esto permitiría añadir notificaciones *Push* nativas, alertando al instante sobre nuevos contratos o mensajes en el chat.

- Expansión del Módulo de Inteligencia Artificial

El Asistente Virtual (basado en Gemini) actualmente sirve como soporte técnico. Como mejora, se podría entrenar al modelo para realizar *Matchmaking Inteligente*: un sistema donde el cliente describe el problema en lenguaje natural (por ejemplo: "*Necesito alguien que cuide a mi abuelo con Alzheimer por las mañanas*") y la IA filtre automáticamente la base de datos y le proponga los 3 perfiles profesionales más adecuados.